

HERDS: Multi-Key Fully Homomorphic Encryption with Sublinear Bootstrapping

Binwu Xiang^{1,2*}, Seonhong Min^{3*}, Intak Hwang³, Zhiwei Wang⁴, Haoqi He⁴, Yuanju Wei⁴, Kang Yang², Jiang Zhang², Yi Deng⁵, and Yu Yu^{6,1}

¹ East China Normal University, Shanghai, China
bwxiang@sc.ecnu.edu.cn

² State Key Laboratory of Cryptology, Beijing, China
{yangk,zhangj}@sklc.org

³ Seoul National University, Seoul, Republic of Korea
{minsh,intak.hwang}@snu.ac.kr

⁴ Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing, China
{wangzhiwei,hehaoqi,weiyuanju}@iie.ac.cn

⁵ School of Cryptology, Xidian University, Xi'an Shannxi, China
ydeng.cas@gmail.com

⁶ Shanghai Jiao Tong University, Shanghai, China
yuyu@yuyu.hk

Abstract. Multi-key fully homomorphic encryption (MK-FHE) enables secure computation over ciphertexts under different keys, but its practicality is hindered by inefficient bootstrapping. In this work, we propose HERDS, a new MK-FHE scheme with highly efficient bootstrapping. Our bootstrapping framework improves upon the best-known complexity, reducing it from $O(dkn)$ to $O(kn)$, and further to $O(\sqrt{kn})$ under parallelization, where d is the gadget length (typically scaling with the number of parties k) and n is the LWE dimension. The framework consists of two main components: (i) a ciphertext conversion algorithm that transforms a multi-key LWE ciphertext into k vectorized RLWE ciphertexts via k optimized blind rotations and dk key-switching operations, and (ii) a hybrid accumulator that aggregates these into a single multi-key RLWE ciphertext. We implemented HERDS on both CPU and GPU platforms to demonstrate its practicality. For $k = 16$, we achieve $3.3\times$ and $7.2\times$ improvements on CPU, compared to the state-of-the-art schemes by Kwak et al. (PKC 2024) and by Xiang et al. (ASIACRYPT 2024), respectively. We further achieve a $195\times$ GPU acceleration, compared to our CPU runtime. As a byproduct, we design a new distributed-decryption protocol, which allows us to obtain a ciphertext with a small noise bound, and thus does not blow up the parameters.

Keywords: MK-FHE · Bootstrapping · Blind Rotation · Hardware Accelerator.

* These authors contributed equally.

1 Introduction

Fully homomorphic encryption (FHE) serves as a powerful cryptographic primitive for enabling arbitrary computations on encrypted data without necessitating decryption, and its utility has led to extensive adoption across diverse domains, including privacy-preserving machine learning [35,42,36,29,21], private information retrieval [3,44,23], private set intersection [13,12,19], and oblivious message retrieval [39,40,37]. However, conventional single-key FHE schemes [25,7,8,26,15,17] require all inputs to be encrypted under the same secret key, rendering them unsuitable for the multi-user setting where data owners require independent key ownership. To address this limitation, two related primitives have been widely explored: *multi-party FHE* (MP-FHE) and *multi-key FHE* (MK-FHE). MP-FHE [4,47,48,51,54,57,45,20] relies on a collaborative setup phase in which participants jointly generate keys, enabling efficient ciphertext operations with reduced overhead. However, it requires all parties to be known and fixed during setup, limiting flexibility in dynamic or asynchronous environments. In contrast, MK-FHE allows users to independently generate their own key pairs and participate in computations without prior coordination. This ad-hoc model supports dynamic participation and avoids interactive setup, at the expense of increased computational and storage costs. Early MK-FHE schemes, due to their substantial ciphertext expansion and high complexity in bootstrapping, mostly remained at the theoretical level [41,18,49,55,14] or were only implemented for a leveled HE version [11,31]. Therefore, in this work, we focus on TFHE-like MK-FHE schemes, the only known constructions that implement the bootstrapping algorithm, which were first proposed in [10] and subsequently improved by [33].

Bootstrapping is the most complex and computationally intensive component in the entire MK-FHE construction. At its core, the operation homomorphically evaluates the decryption function of a multi-key ciphertext. Specifically, given a multi-key LWE ciphertext of the form $\bar{\mathbf{c}} = (b, \mathbf{a}_1, \dots, \mathbf{a}_k) \in \mathbb{Z}_q^{kn+1}$ encrypted under the collective secret key $\bar{\mathbf{z}} = (\mathbf{z}_1, \dots, \mathbf{z}_k)$ where k is the number of parties and each $\mathbf{z}_i \in \mathbb{Z}^n$ (n being the lattice dimension) belongs to the i -th party, bootstrapping aims to homomorphically compute the inner product $\text{Dec}(\bar{\mathbf{c}}, \bar{\mathbf{z}}) = b + \sum_{i=1}^k \langle \mathbf{a}_i, \mathbf{z}_i \rangle \bmod q$. The primary challenge of this process is typically the homomorphic evaluation of the modular reduction function, which is inherently non-linear. This complexity is substantially mitigated by an algebraic insight: when instantiated in the ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ where $q = 2N$, the multiplicative order of X becomes exactly q . This fundamental property induces an implicit modular reduction in the exponent, yielding the critical equivalence: $X^{b + \sum_{i=1}^k \langle \mathbf{a}_i, \mathbf{z}_i \rangle} = X^{b + \sum_{i=1}^k \langle \mathbf{a}_i, \mathbf{z}_i \rangle \bmod q}$. Thus, while the mod q is still mathematically present, its effect is absorbed by the ring structure, and it does not incur additional computational cost. As a result, the bootstrapping task reduces to homomorphically computing the linear term $\sum_{i,j} c_{i,j} z_{i,j}$ in the exponent. This operation is referred to as *blind rotation* [10,33,59], and it remains the most computationally intensive component in the bootstrapping. In the seminal RLWE-based construction by Chen, Chillotti, and Song [10], blind rotation was

achieved via a novel *hybrid product* operation, achieving a computational complexity of $O(k^2n)$. This was significantly improved by Kwak, Min, and Song [33], whose refined approach decomposes the multi-key blind rotation into single-key operations, reducing the complexity to $O(dkn)$, where $d = O(\log k + \log n)$. This incurs a quasi-linear computation cost in k or n .

Despite significant advances in MK-FHE, its practical adoption remains constrained by fundamental computational inefficiencies. As shown before, while the asymptotic complexity of the state-of-the-art bootstrapping algorithms is bounded by $O(dkn)$, resulting in quasi-linear complexity, practical parameter choices require $d \geq 3$ with further increases as k and n grow. This is reflected in concrete performance: an eight-party bootstrapping operation currently requires over 760 ms [59], compared to merely 10 ms for single-key bootstrapping [28,9] at 128-bit security, a gap that widens substantially as more parties join. Moreover, a critical obstacle to real-world deployment is the inherently low parallelizability of core homomorphic operations, especially the blind rotation which requires sequential homomorphic accumulation over each component of the n -dimensional secret key, severely limiting abilities for parallel execution. This stands in stark contrast to modern computing paradigms, where applications are rarely run on single-threaded CPUs and instead leverage multi-core architectures, GPUs, or specialized hardware to maximize throughput.

Another challenge in the construction of a multi-key variant of FHE schemes is the distributed decryption. After the computation, all the parties need to collaboratively decrypt the output ciphertext since it is encrypted under a joint secret consisting of all parties' secrets. There are two major approaches to achieve the distributed decryption of an LWE ciphertext, namely the MPC-based approach and the partial-decryption based approach. The MPC-based approach directly calls the existing MPC protocols to evaluate the decryption circuit on the input LWE ciphertext [32,33]. While this approach is conceptually easier since we can use a generic MPC protocol as an underlying solution, it requires high rounds of interaction or a large communication cost, and is not practical for real-world applications. On the other hand, the partial-decryption based approach requires each party to use its individual secret key to compute a partial decryption, and these results are combined to reconstruct the message. Being not dependent on other MPC primitives, this approach is the primary approach for distributed decryption in the FHE setting. Roughly speaking, given the MK-LWE ciphertext $(b, \mathbf{a}_1, \dots, \mathbf{a}_k) \in \mathbb{Z}_q^{kn+1}$ under the joint secret $\bar{\mathbf{z}} = (\mathbf{z}_1, \dots, \mathbf{z}_k) \in \mathbb{Z}^{kn}$, each i -th party outputs the partial decryption $d_i := \langle \mathbf{a}_i, \mathbf{z}_i \rangle + f_i \pmod{q}$ for the additional noise f_i . Next, each party can compute $b + \sum_{i=1}^k d_i = \lfloor \frac{q}{t} \rfloor m + e + \sum_{i=1}^k f_i \pmod{q}$ to reconstruct the phase of the input ciphertext and successfully decrypt the input ciphertext, where m and e denote the message and noise of the input ciphertext, respectively, and t is the message space. However, applying this to TFHE-like schemes can be quite challenging. To guarantee security, the additional flooding noise f_i should be exponentially larger than the input noise e [25]. Considering that the TFHE performs over a minimal and compact parameter set, applying this to TFHE may lead to a considerably slow latency of

the bootstrapping due to larger parameter sets. To mitigate this, Dahl et al. [20] proposed applying the heavy bootstrapping only for the decrypted ciphertext, while maintaining a tight and small parameter for the circuit evaluation, under the threshold setting. However, this strategy still requires a time-consuming process of slow bootstrapping under a large parameter, especially in the multi-key setting.

1.1 Our Results

In this work, we present **HERDS**, a new MK-FHE scheme with highly efficient bootstrapping and distributed decryption. Our bootstrapping framework reduces the complexity of prior constructions from $O(dkn)$ to $O(kn)$, and further to $O(\sqrt{nk})$ under parallelization. It consists of two layers: (i) ciphertext conversion, which transforms a multi-key LWE ciphertext into k vectorized RLWE ciphertexts via k optimized blind rotations and kd key-switching operations; and (ii) a hybrid accumulator that aggregates these k ciphertexts into a single multi-key RLWE ciphertext. For decryption, we design a distributed protocol that simulates blind rotation by sequentially rotating predetermined LWE ciphertexts with only the noise of a single public-key encryption, thereby reducing the ciphertext modulus and ring degree from $(q, N) = (2^{128}, 4096)$ in [20] to $(2^{64}, 2048)$, which match our bootstrapping parameters and achieve significantly better performance.

In Table 1, we compare state-of-the-art RLWE-based MKFHE schemes [10, 33] and NTRU-based MKFHE schemes [59, 52] with respect to two key metrics: computational complexity, measured in gadget products [33], and output noise magnitude. Here n is the lattice dimension, k the number of parties (with $k < n$), and d the gadget length, typically $d = O(\log k + \log n)$. It is easy to see that **HERDS** achieves the best-known bootstrapping complexity in terms of both sequential and parallel executions.

Table 1. Theoretical comparison of MK-TFHE schemes. Green marks the best asymptotic bounds achieved by prior schemes, while red marks the bounds achieved by **HERDS**.

Scheme	Assumption	Comput. Complexity		Noise
		Serial	Parallel	
CCS [10]	RLWE	$O(k^2n)$	$O(kn)$	$\text{poly}(n, k)$
KMS [33]	RLWE	$O(dkn)$	$O(n + k)$	$\text{poly}(n, k)$
XZW+ [59]	NTRU	$O(k^2n)$	$O(kn)$	$\text{poly}(n, k)$
FINALLY [52]	NTRU	$O(kn)$	$O(kn)$	$\exp(k)$
HERDS	RLWE	$O(kn)$	$O(\sqrt{kn})$	$\text{poly}(n, k)$

In Table 2, we benchmark our bootstrapping against the state-of-the-art RLWE-based scheme KMS (PKC 2024) [33] and the NTRU-based scheme XZW+

(ASIACRYPT 2024) [59].⁷ We exclude CCS [10] since KMS already outperforms it, and FINALLY [52] as it provides only theoretical estimates. Following KMS, we adopt parameters at the 100-bit security for a fair comparison, and additionally report results for 128-bit security in Table 4, Sec. 3.4. Our implementation on both CPU and GPU shows substantial improvements: for $k \in \{2, \dots, 16\}$, our scheme achieves $1.9\times$ – $3.3\times$ faster runtime than KMS (PKC 2024) and $1.7\times$ – $7.2\times$ faster than XZW+ (ASIACRYPT 2024) on CPU, while delivering $50\times$ – $338\times$ acceleration on GPU relative to our own CPU baseline for $k \in \{2, \dots, 32\}$. The advantage becomes increasingly pronounced as the number of parties k grows.

Table 2. Bootstrapping runtime (in milliseconds).

Scheme	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
KMS [33] (CPU)	82 (1.9 $\times$)	255 (3.0 $\times$)	624 (3.4 $\times$)	1541 (3.3 $\times$)	4512 (3.8 $\times$)
XZW+ [59] (CPU)	75 (1.7 $\times$)	252 (2.9 $\times$)	899 (5.0 $\times$)	3377 (7.2 $\times$)	–
HERDS (CPU)	43 (1.0$\times$)	86 (1.0$\times$)	181 (1.0$\times$)	471 (1.0$\times$)	1195 (1.0$\times$)
HERDS (GPU)	0.86	1.18	1.78	2.42	3.54

1.2 Technical Overview

Before delving into the technical ingredients of our construction, we briefly review prior approaches of RLWE-based multi-key bootstrapping. While there have been several attempts to extend the single-key TFHE to the multi-key setting [10, 33, 1, 60, 59], we focus on Kwak, Min, and Song’s approach [33], which provides the best computational complexity without the NTRU assumption and with the smallest setup communication cost. The central technical challenge in multi-key bootstrapping lies in blind rotation, as it requires an external product in a multi-key manner. This was first addressed in [10] and later refined in [33] via a new *uni-encryption* scheme and a corresponding multiplication algorithm between a multi-key RLWE ciphertext and a uni-encryption, termed the *hybrid product*. The very first construction by Chen et al. [10] was to simply replace all the external products in single-key TFHE [17] with hybrid products, which leads to a heavy blind rotation step with asymptotic time complexity of $O(k^2n)$ growing quadratically with respect to the number of parties k .

Kwak et al. [33] improved this approach by replacing the sequential blind rotation with k parallel single-key blind rotations.⁸ Specifically, given a multi-key RLWE encryption $(b, \mathbf{a}_1, \dots, \mathbf{a}_k) \in \mathbb{Z}_q^{kn+1}$ under the joint key $(\mathbf{z}_1, \dots, \mathbf{z}_k)$ where each $\mathbf{z}_i \in \mathbb{Z}^n$ belongs to the i -th party, each party i generates a set of d RLWE ciphertexts (also referred to as an RLEV encryption) where the j -th ciphertext

⁷ We use the gadget product as the basic unit, as in [33]. One gadget product equals d ring multiplications.

⁸ For completeness, we review the single-key blind rotation in full version.

encrypts $g_j X^{\langle \mathbf{a}_i, \mathbf{z}_i \rangle}$ for $0 \leq j < d$ for some gadget vector $(g_0, \dots, g_{d-1})$. Note that this can be done by setting the accumulator trivial encryptions of each entry of the gadget vector, and running the blind rotation algorithm. Then the resulting RLWE ciphertexts form an RLEV encryption of $X^{\langle \mathbf{a}_i, \mathbf{z}_i \rangle}$ naturally. The procedure requires kd single-key blind rotations, corresponding to a total complexity of $O(kdn)$ gadget products. Subsequently, these RLEV encryptions are multiplied with the trivial multi-key RLWE encryption of $-[q/8]X^{-N/2+b} \cdot \sum_{i=0}^{N-1} X^i$ using an ingenious multiplicative operation between a (non-fresh) single-key RLEV encryption and a multi-key RLWE encryption, termed the *generalized external product*. A single generalized external product incurs a cost of $O(k)$. Since the procedure requires aggregating k RLEV ciphertexts, the total cost of this step is $O(k^2)$, which is asymptotically dominated by the blind rotation cost under typical parameter choices where $k \ll n$. Thus, the overall complexity of the blind rotation step is $O(kdn + k^2) = O(kdn)$. After blind rotation, the procedure continues with coefficient extraction and key-switching on the resulting multi-key ciphertext. These steps are essentially identical to their single-key counterparts and are asymptotically negligible compared to the multi-key blind rotation. See [33] for further details.

Our Approach. Our first observation is that although the construction of Kwak et al. [33] improves the asymptotic complexity, it requires a larger ring dimension ($N = 2048$ instead of 1024) to control noise. This increase inherently supports higher message precision. Inspired by the leveled homomorphic circuits of [16], we adopt a horizontal packing-like technique to jointly evaluate all d blind rotation outputs. Specifically, we initialize the test vector as $\sum_{j=0}^{d-1} g_j X^j$ and, after blind rotation for each $\mathbf{a}_i$, obtain an RLWE ciphertext encrypting $\sum_j g_j X^{j+\langle \mathbf{a}_i, \mathbf{s}_i \rangle}$. Consequently, only k single-key blind rotations are required for the entire multi-key bootstrapping, yielding complexity $O(kn)$. The remaining task is to decode $\sum_j g_j X^{j+\langle \mathbf{a}_i, \mathbf{s}_i \rangle}$ into the desired RLEV ciphertext.

Assume $d \mid \langle \mathbf{a}_i, \mathbf{s}_i \rangle$ for some power-of-two d .⁹ We propose a homomorphic decoding algorithm that decomposes the initial ciphertext

$$c^{(0)} = \text{RLWE} \left(\sum_{j=0}^{d-1} g_j X^{j+\langle \mathbf{a}_i, \mathbf{s}_i \rangle} \right)$$

into d ciphertexts, with the j -th output equal to $\text{RLWE}(g_j X^{\langle \mathbf{a}_i, \mathbf{s}_i \rangle})$. The algorithm proceeds recursively for $\ell = \log_2 d$ stages in a binary tree. At level i , each ciphertext $c_j^{(i)}$ produces two children via homomorphic automorphisms and key-switching:

$$c_1 \leftarrow c_j^{(i)} \cdot X^{-2^i}, \quad \tilde{c}_0 \leftarrow \text{KS}(\phi_t(c_j^{(i)})), \quad \tilde{c}_1 \leftarrow \text{KS}(\phi_t(c_1)),$$

⁹ This can be enforced by a tailored modulus switching: for each $a_{i,j} \in \mathbf{a}_i$, compute $\left\lfloor \left[\frac{a_{i,j} \cdot 2N}{dq} \right] \cdot d \right\rfloor_{2N}$.

with $t = N/2^i + 1$. The updated ciphertexts are

$$c_j^{(i+1)} \leftarrow c_j^{(i)} + \tilde{c}_0, \quad c_{j+2^i}^{(i+1)} \leftarrow c_1 + \tilde{c}_1,$$

where $\phi_t : X \mapsto X^t$ permutes coefficients, and KS restores the original key. After ℓ stages, the d leaves $\{c_j^{(\ell)}\}_{j=0}^{d-1}$ isolate the terms $\text{RLWE}(g_j X^{\langle \mathbf{a}_i, \mathbf{s}_i \rangle})$.

The decoding requires only d key-switching operations, with complexity $O(d)$. Together with the $O(kn)$ cost of blind rotations, the total complexity is $O(kn + kd)$. Since d is typically constant relative to n , the overall complexity simplifies to $O(kn)$.

Making it More Parallel-Friendly We observe that although the previous construction reduces the complexity to $O(kn)$, the factor n remains dominant since blind rotation inherently requires n serial iterations, which severely restricts parallelism. To address this limitation, we revisit the structure of the vectors $\mathbf{a}_i = (a_{i,0}, \dots, a_{i,n-1})$ in the MK-LWE ciphertext. The key idea is to reinterpret this one-dimensional vector through a tensor decomposition of its index space. By factoring the dimension as $n = \omega_1 \cdot \omega_2$, we obtain an isomorphism that represents $\mathbf{a}_i$ as a matrix in $\mathbb{Z}^{\omega_2 \times \omega_1}$, naturally inducing the block decomposition

$$\mathbf{a}_i = \mathbf{S}_{i,1} \parallel \dots \parallel \mathbf{S}_{i,\omega_2}, \quad \mathbf{S}_{i,\ell} = (a_{i,(\ell-1)\omega_1}, \dots, a_{i,\ell\omega_1-1}).$$

This representation enables block-parallel ciphertext conversion: each block $\mathbf{S}_{i,\ell}$ can be processed independently through a length- ω_1 homomorphic accumulation. In addition, we design a parallel-friendly homomorphic decoding algorithm requiring only $\log d$ parallelizable key-switching operations (see Alg. 3).

By combining these two layers of parallelism, a single party can generate ω_2 independent RLEV ciphertexts

$$\{C_{i,\ell} \in R_q \mid \ell = 1, \dots, \omega_2\},$$

where each $C_{i,\ell}$ encodes the contribution of its block:

$$C_{i,\ell} = X^{\sum_{j=0}^{\omega_1-1} a_{i,(\ell-1)\omega_1+j} \cdot Z_{i,(\ell-1)\omega_1+j}}.$$

This two-level structure provides an ω_2 -fold amortization of the blind-rotation cost. Under parallelization, the procedure generates $k\omega_2$ RLEV ciphertexts with complexity $O(\omega_1)$, followed by hybrid accumulation (Alg. 4). As analyzed in Sec. 3.3, the hybrid accumulation operation is parallelizable; each such operation has constant cost under parallel execution. Thus, the resulting parallel complexity remains $O(k\omega_2 + \omega_1)$, minimized at $\omega_1 \approx \sqrt{nk}$ and $\omega_2 \approx \sqrt{n/k}$, yielding $O(\sqrt{nk})$.

Efficient Distributed Decryption Protocol. The main idea is to homomorphically decrypt the input ciphertext with only additive operations, in a similar manner to blind rotation. To be specific, given a multi-key TFHE ciphertext $\mathbf{c} = (b, \mathbf{a}) = (b, \mathbf{a}_1, \dots, \mathbf{a}_k) \in \mathbb{Z}_q^{kn+1}$ under the joint secret $\bar{\mathbf{z}} = (\mathbf{z}_1, \dots, \mathbf{z}_k) \in \mathbb{Z}^{kn}$,

we prepare q trivial encryptions ACC_i of $\lfloor 2/q \cdot i \rfloor$. Observe that the encrypted message of $\mathbf{c}$ can be obtained by decrypting $\text{ACC}_{2(b+\langle \mathbf{a}, \mathbf{z} \rangle) \pmod{q}}$, our aim is to collectively select the right $\text{ACC}_{2(b+\langle \mathbf{a}, \mathbf{z} \rangle) \pmod{q}}$ among the q ciphertexts. To achieve this goal, we first apply cyclic permutation on ciphertexts, by the index $2b$. In other words, we rotate the ciphertexts so that the accumulators are re-ordered in $\text{ACC}_{2b}, \text{ACC}_{2b+1}, \dots, \text{ACC}_{2b-1}$. Then, similarly, each i -th party rotates it by the index $\text{ACC}_{2\langle \mathbf{a}_i, \mathbf{z}_i \rangle \pmod{q}}$ sequentially. However, a naïve approach may leak secret information on $\mathbf{z}_i$ since an adversary can recover the rotation index $2\langle \mathbf{a}_i, \mathbf{z}_i \rangle \pmod{q}$. Therefore, each party needs to hide their rotation index by adding public-key encryptions to the accumulators. At the end of the computation, it is easy to show that the first accumulator is essentially an aggregation of $\text{ACC}_{2(b+\langle \mathbf{a}, \mathbf{z} \rangle) \pmod{q}}$ and k public key zero encryptions, hence having a small noise of only k times public key encryption. Therefore, we can take the first accumulator and perform a distributed flood-based decryption to recover the message. As a result, the ciphertext modulus can be compact enough to accommodate just the flooding and public key encryption noise.

Although this approach guarantees small modulus consumption, it requires a large communication cost to send q ciphertexts. Therefore, we propose to utilize the modulus switching technique to minimize the number of accumulators. Roughly speaking, we redefine the i -th accumulator ACC_i to be initialized as a trivial encryption of $\lfloor 2i/r \rfloor$. Then, instead of directly computing $\lfloor (r/q) \cdot c \rfloor$ and executing the algorithm, the parties rotate the ciphertexts by the index $\lfloor (r/q) \cdot 2\langle \mathbf{a}_i, \mathbf{z}_i \rangle \rfloor \pmod{r}$, using accumulators that have been pre-rotated by the index $\lfloor (r/q) \cdot 2b \rfloor$. Further details are provided in Sec. 4.

1.3 Related Works

Since the seminal work by Chen, Chillotti, and Song [10], which first extended the single-key TFHE scheme to the multi-key setting, a series of subsequent studies have been proposed. In particular, the approach of Kwak, Min, and Song [33] significantly improved the computational efficiency of MK-TFHE bootstrapping by optimizing the evaluation flow, thereby reducing the complexity from quadratic to linear with respect to the number of parties.

Akin et al. [1] proposed a dynamic variant of TFHE in the multi-party setting, whose sequential complexity is comparable to ours but is inherently difficult to parallelize. Moreover, their construction requires interactive communication among the parties to jointly generate evaluation keys prior to computation, which deviates from the standard MK-FHE framework where evaluation keys can be generated non-interactively. As a result, their scheme is more appropriately classified as multi-party FHE, and thus it is not directly comparable to our MK-FHE design.

Several NTRU-based MK-TFHE schemes have been proposed to improve bootstrapping efficiency. Xu et al. [60] relied on highly sparse secret keys with fixed Hamming weight; however, subsequent analysis showed that such sparsity introduces security vulnerabilities [30]. Park et al. [52] proposed another

variant whose noise grows exponentially with the number of parties, and no implementation was provided. Xiang et al. [59] adapted the Chen–Chillotti–Song framework [10] to NTRU, yet the cost remains quadratic in the number of parties. Moreover, Ducas and van Woerden [22] showed that to resist sublattice attacks the modulus must stay below the fatigue point $n^{2.484+o(1)}$, a stringent constraint that makes parameter selection for larger-party settings (e.g., > 16) highly restrictive.

Comparison with threshold FHE. Threshold FHE (ThFHE) generally has a larger round complexity as its setup is interactive and does not allow parties to participate dynamically. In contrast, MK-FHE requires no interactive setup: each user locally generates a public key and encrypts messages under the public key. Computation can then be carried out directly over ciphertexts tied to different keys. This removes the need for parties to coordinate or be online at the same time to set up a common key, making MK-FHE well-suited to distributed and asynchronous deployments. Moreover, existing threshold variants of TFHE either rely on a generic MPC for distributed key generation or report $1.9\text{--}4.7\times$ higher latency ([1,38,53]) than our MK-FHE scheme. In either case, they remain hard to parallelize, and thus the computational complexity still scales linearly in the ciphertext dimension n . In contrast, our approach is highly parallel-friendly and can achieve sublinear complexity.

2 Preliminaries

2.1 Notation

Let $\mathbb{R}$ (resp. $\mathbb{Z}$) denote the set of real numbers (integers). Vectors are denoted by bold lowercase letters (e.g., $\mathbf{a}$, $\mathbf{b}$), and matrices by bold uppercase boldface letters (e.g., $\mathbf{A}$, $\mathbf{B}$). The inner product of vectors $\mathbf{a}$ and $\mathbf{b}$ is written as $\langle \mathbf{a}, \mathbf{b} \rangle$. The index set $[k]$ denotes $\{1, 2, \dots, k\}$. For a positive integer q and a power-of-two N , define the $2N$ -th cyclotomic ring $R = \mathbb{Z}[X]/(X^N + 1)$ and its quotient ring $R_q = R/qR$. Let $\phi_t : R \rightarrow R$ denote the ring homomorphism defined by mapping a polynomial $a(X)$ to $a(X^t)$. For any pair $c = (a, b) \in R^2$, we extend ϕ_t component-wise via $\phi_t(c) := (\phi_t(a), \phi_t(b))$.

$\mathcal{U}(\mathcal{S})$ denotes the uniform distribution from a countable set $\mathcal{S}$. We write $\mathcal{D}_\alpha$ and $\mathcal{D}_\beta$ to denote that rounded Gaussian with standard deviation α over $\mathbb{Z}$, and the coefficient-wise rounded Gaussian with standard deviation β over R . By $\|\cdot\|_1$ and $\|\cdot\|_\infty$, we denote the ℓ_1 and ℓ_∞ norms. By $\lceil \cdot \rceil$, $\lfloor \cdot \rfloor$, and $\text{round}(\cdot)$, we denote the ceiling, floor, and round functions, respectively. Finally, $[\cdot]_q$ denotes the modulo reduction by q .

2.2 Gadget Decomposition

Gadget decomposition constitutes a fundamental technique in fully homomorphic encryption schemes, providing a crucial balance between noise management and computational efficiency.

Let $q, d > 0$ be integers. If the vector $\mathbf{g} = (g_0, \dots, g_{d-1}) \in \mathbb{Z}^d$ and a function $\mathbf{g}^{-1} : R_q \rightarrow R^d$ satisfy the following conditions for any $a \in R_q$, then we call $\mathbf{g}$ a gadget vector, and $\mathbf{g}^{-1}$ a gadget decomposition corresponding to $\mathbf{g}$.

- (i) For $\mathbf{g}^{-1}(a) = (a_0, \dots, a_{d-1})$, $\|a_i\|_\infty \leq B/2$ for all $0 \leq i < d$.
- (ii) $\|[a - \langle \mathbf{g}, \mathbf{g}^{-1}(a) \rangle]_q\|_\infty \leq \epsilon$ for some fixed real $\epsilon > 0$.

One concrete example of the gadget decomposition is the digit decomposition; each entry of the gadget vector $\mathbf{g}$ is given by $g_i = q/B^{d-i}$ assuming that B^d divides q . Then, it satisfies that $\epsilon = \frac{q}{2B^d}$. In the paper, the gadget vector and its decomposition function can be determined by two parameters B and d . In the paper, we only consider the digit decomposition. It is worth noting that if a follows the uniform distribution over R_q , then the coefficient of each entry a_i of $\mathbf{g}^{-1}(a)$ and $[a - \langle \mathbf{g}, \mathbf{g}^{-1}(a) \rangle]_q$ follows uniform distributions over the set $\mathbb{Z} \cap [-B/2, B/2]$ and $\mathbb{Z} \cap [-\frac{q}{2B^d}, \frac{q}{2B^d}]$, respectively, hence having variances of $\frac{B^2}{12}$ and $\frac{\epsilon^2}{3} = \frac{q^2}{12B^{2d}}$.

2.3 Multi-key Fully Homomorphic Encryption

Let $\mathcal{M}$ denote the message space with an associated arithmetic structure, and let k be the upper bound on the number of parties. A MK-FHE scheme consists of a tuple of PPT algorithms (Setup, KeyGen, Enc, Dec, Eval) with the following functionality:

- **Setup**(1^λ): Given the security parameter λ , output the parameter set pp .
- **KeyGen**(i): For each party i , generate a public/secret key pair (pk_i, sk_i) .
- **Enc**($pk_i; m$): Given a message $m \in \mathcal{M}$ and a public key pk_i , output a ciphertext ct_i .
- **Eval**($\mathcal{C}, \{pk_i\}_{i \in [k]}; \{\overline{ct}_j\}_{j \in [\ell]}$): Given a circuit $\mathcal{C}$, a tuple of ciphertexts $\{\overline{ct}_j\}_{j \in [\ell]}$, and the set of public keys $\{pk_i\}_{i \in [k]}$, output an evaluated ciphertext $\overline{ct}$.
- **Dec**($\{sk_i\}_{i \in [k]}; \overline{ct}$): Given a ciphertext $\overline{ct}$ and the secret keys $\{sk_i\}_{i \in [k]}$, output a message m .

A MK-FHE scheme is *compact* if there exists a polynomial function $\text{poly}(\cdot, \cdot)$ such that the length of a ciphertext associated with k keys is at most $\text{poly}(\lambda, k)$. A multi-key FHE scheme is said to be *semantically secure* if its encryption satisfies IND-CPA security [33].

Let $\{\overline{ct}_j\}_{j \in [\ell]}$ be ciphertexts encrypting messages $m_1, \dots, m_\ell$, respectively, under a set of public keys $\{pk_i\}_{i \in [k]}$. The scheme is *correct* if the following holds with overwhelming probability:

$$\text{Dec}(\{sk_i\}_{i \in [k]}; \text{Eval}(\mathcal{C}, \{pk_i\}_{i \in [k]}; \{\overline{ct}_j\}_{j \in [\ell]})) = \mathcal{C}(m_1, \dots, m_\ell).$$

2.4 Hardness Problems and Ciphertexts

Definition 1 (The decisional LWE Problem [56]). Let q, n be positive integers, ψ be a secret key distribution over $\mathbb{Z}$, and χ be a noise distribution over $\mathbb{Z}$. The LWE problem, denoted by $\text{LWE}_{n,q,\psi,\chi}$, asks an adversary $\mathcal{A}$ to distinguish the following two cases:

- $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{z} \rangle + e \pmod{q})$ where $\mathbf{a} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$, $\mathbf{z} \leftarrow \psi^n$, $e \leftarrow \chi$,
- $(\mathbf{u}, v)$ where $\mathbf{u} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$, $v \leftarrow \mathcal{U}(\mathbb{Z}_q)$.

The $\text{LWE}_{n,q,\psi,\chi}$ assumption states that any polynomial-time adversary $\mathcal{A}$ achieves negligible advantage in distinguishing these distributions.

Based on the decisional LWE assumption, the single-key and multi-key LWE encryption schemes are described as follows.

- **Single-Key LWE Ciphertext:** The LWE encryption of a message $m \in \mathbb{Z}_q$ under the secret $\mathbf{z} \leftarrow \psi^n$ is given by the pair $(b, \mathbf{a}) \in \mathbb{Z}_q^{n+1}$ such that $\varphi_{\mathbf{z}}(m) := b + \langle \mathbf{a}, \mathbf{z} \rangle = m + e \pmod{q}$, for some small e being the noise term.
- **Multi-Key LWE Ciphertext:** For k secret keys $\mathbf{z}_1, \dots, \mathbf{z}_k \leftarrow \psi^n$, the multi-key LWE encryption of a message $m \in \mathbb{Z}_q$ under the joint secret $\bar{\mathbf{z}} = (\mathbf{z}_1, \dots, \mathbf{z}_k)$ is given by the tuple $(b, \mathbf{a}_1, \dots, \mathbf{a}_k) \in \mathbb{Z}_q^{kn+1}$ such that $\varphi(m)_{\bar{\mathbf{z}}} = b + \sum_{i=1}^k \langle \mathbf{a}_i, \mathbf{z}_i \rangle = m + e \pmod{q}$, for some small e being the noise term.

Definition 2 (The decisional RLWE Problem [43]). Let q, N be positive integers, ψ' be a secret key distribution over R_q , and χ' be a noise distribution over R_q . The RLWE problem, denoted by $\text{RLWE}_{N,q,\psi',\chi'}$, asks an adversary $\mathcal{A}$ to distinguish between the following two distributions:

- $(a, b = a \cdot s + e \pmod{q})$ where $a \leftarrow \mathcal{U}(R_q)$, $s \leftarrow \psi'$, and $e \leftarrow \chi'$;
- (u, v) where $u, v \leftarrow \mathcal{U}(R_q)$.

The $\text{RLWE}_{N,q,\psi',\chi'}$ assumption states that any polynomial-time adversary $\mathcal{A}$ achieves negligible advantage in distinguishing these distributions.

Similar to LWE, the single-key and multi-key RLWE encryption schemes are described as follows.

- **Single-Key RLWE Ciphertext:** The RLWE encryption of a message $\mu \in R_q$ under the secret $s \leftarrow \psi'$ is given by a pair $c = (b, a)$ such that $\varphi_s(c) := b + a \cdot s = \mu + e \pmod{q}$, for some small e being the noise term.
- **Multi-Key RLWE Ciphertext:** For k secret keys $s_1, \dots, s_k \leftarrow \psi'$, the multi-key RLWE encryption of a message $\mu \in R_q$ under the joint key $\bar{s} = (s_1, \dots, s_k)$ is given by a tuple $c = (b, a_1, \dots, a_k)$ such that $\varphi_{\bar{s}}(c) := b + \sum_{i=1}^k a_i s_i = \mu + e \pmod{q}$, for some small e being the noise term.

We also describe the gadget-based cryptosystems, namely RLEV and RGSW, and their encryption algorithms below. In the following, we fix the gadget vector $\mathbf{g} = (g_0, \dots, g_{d-1}) \in \mathbb{Z}^d$ and the corresponding decomposition function $h : \mathbb{Z}_q \rightarrow \mathbb{Z}^d$.

- **RLEV.Enc** (μ, s) : Given the message $\mu \in R_q$ and the key $s \in R$, sample $\mathbf{a} \leftarrow \mathcal{U}(R_q^d)$ and $\mathbf{e} \leftarrow \chi^d$. Compute $\mathbf{b} := -\mathbf{a} \cdot s + \mu \cdot \mathbf{g} + \mathbf{e} \pmod{q}$ and output $C := [\mathbf{b} \mid \mathbf{a}] \in R_q^{d \times 2}$.

Note that RLEV ciphertext can be seen as a vector of RLWE ciphertexts $(c_i)_{0 \leq i < d}$ where c_i is an encryption of $\mu \cdot g_i$ under s , for all $0 \leq i < d$. We define

the phase function φ_s for an RLEV ciphertext $C = (c_{i,j})_{0 \leq i < d, 0 \leq j \leq 1} \in R_q^{d \times 2}$ similar to RLWE, as follows:

$$\varphi_s(C) := \begin{bmatrix} c_{0,0} \\ \vdots \\ c_{d-1,0} \end{bmatrix} + \begin{bmatrix} c_{0,1} \\ \vdots \\ c_{d-1,1} \end{bmatrix} \cdot s \pmod{q}.$$

• **RGSW.Enc**(μ, s) : Given the message $\mu \in R_q$ and the key $s \in R$, sample $\mathbf{a} \leftarrow \mathcal{U}(R_q^{2d})$ and $\mathbf{e} \leftarrow \chi^d$. Compute $\mathbf{b} := -\mathbf{a} \cdot s + \mu \cdot \begin{bmatrix} \mathbf{g} & \mathbf{0} \\ \mathbf{0} & \mathbf{g} \end{bmatrix} + \mathbf{e} \pmod{q}$ and output $\overline{C} := [\mathbf{b} \mid \mathbf{a}] \in R_q^{2d \times 2}$.

The RGSW ciphertext can be seen as a vector $(C_0, C_1)^\top$ of RLEV ciphertexts in $R_q^{d \times 2}$, where C_0 and C_1 are encryptions of μ and $\mu \cdot s$ under the secret s , respectively. It is also worth noting that different gadget parameters can be used for the RLEV ciphertexts composing the RGSW ciphertext, as proposed in [6]. We also define the phase function φ_s for an RGSW ciphertext $\overline{C} = (c_{i,j})_{0 \leq i < 2d, 0 \leq j \leq 1}$ as follows:

$$\varphi_s(\overline{C}) := \begin{bmatrix} c_{0,0} \\ \vdots \\ c_{2d-1,0} \end{bmatrix} + \begin{bmatrix} c_{0,1} \\ \vdots \\ c_{2d-1,1} \end{bmatrix} \cdot s \pmod{q}.$$

Below, we describe the operations between RLWE-based ciphertexts and provide an noise analysis. In the analysis, we fix the key distribution ψ' of RLWE-based encryptions to be a binary distribution over R .

Definition 3 (Scalar RLEV Multiplication). Let $a \in R_q$ be a polynomial and C be an RLEV ciphertext of the message $\mu \in R_q$, associated with a gadget vector $\mathbf{g} \in \mathbb{Z}^d$ and decomposition function $\mathbf{g}^{-1}$. Then, the scalar RLEV multiplication, denoted by $\odot : R_q \times R_q^{d \times 2} \rightarrow R_q^2$ is defined as $a \odot C := \mathbf{g}^{-1}(a)^\top \cdot C$.

Lemma 1 (Scalar RLEV Multiplication Noise). Let $a \in R_q$ be a uniform random scalar polynomial and $C \in R_q^{d \times 2}$ be an RLEV encryption of $\mu \in R_q$ with gadget parameters d and B , and a noise variance σ_{in}^2 under secret s . The scalar RLEV multiplication $a \odot C$ outputs an encryption $c \in R_q^2$ such that $\varphi_s(c) = \mu \cdot a + e \pmod{q}$ for some small noise e , whose variance $\sigma_\odot^2$ is bounded by

$$\sigma_\odot^2 \leq dN \cdot \frac{B^2}{12} \cdot \sigma_{in}^2 + \|\mu\|_1 \cdot \frac{\epsilon^2}{3},$$

where $\epsilon = \frac{q}{B^d}$.

We defer the proof to Appendix B.1

Definition 4 (RLWE-RGSW Multiplication). Let $c = (b, a) \in R_q^2$ be an RLWE ciphertext, and $\overline{C} \in R_q^{2d \times 2}$ be an RGSW ciphertext of message μ under the secret s , associated with the gadget vector $\mathbf{g} \in \mathbb{Z}^d$ and decomposition function

$\mathbf{g}^{-1}$. Then, the RLWE-RGSW multiplication $\boxdot : R_q^2 \times R_q^{2d \times 2} \rightarrow R_q^2$ is defined as:

$$c \boxdot \overline{C} = \begin{bmatrix} \mathbf{g}^{-1}(b) \\ \mathbf{g}^{-1}(a) \end{bmatrix}^\top \cdot \overline{C} \pmod{q}.$$

Throughout the paper, we abuse the notation $\mathbf{g}^{-1}$ to denote the element-wise decomposition of a vector. In other words, for an RLWE ciphertext $c = (b, a) \in R_q^2$, $\mathbf{g}^{-1}(c) = \begin{bmatrix} \mathbf{g}^{-1}(b) \\ \mathbf{g}^{-1}(a) \end{bmatrix} \in R_q^{2d}$.

Lemma 2 (RLWE-RGSW Multiplication Noise). *Let $c = (b, a) \in R_q$ be an RLWE encryption of $m \in R_q$ with a noise variance of σ_{RLWE}^2 under s , and $\overline{C}$ be an RGSW encryption of μ , with gadget parameters d and B and a noise variance of σ_{RGSW}^2 under secret s . Assuming that the RLWE ciphertext looks uniformly distributed over R_q^2 , the RLWE-RGSW multiplication $c \boxdot \overline{C}$ outputs an encryption $c' \in R_q^2$ such that $\varphi_s(c') = \mu \cdot m + e \pmod{q}$ for some small e , whose variance $\sigma_{\boxdot}^2$ is bounded by*

$$\sigma_{\boxdot}^2 \leq \|\mu\|_1 \cdot \sigma_{RLWE}^2 + dN \frac{B^2}{6} \sigma_{RGSW}^2 + (\|\mu\|_1 + \|\mu s\|_1) \frac{\epsilon^2}{3}$$

where $\epsilon = \frac{q}{B^d}$.

We defer the proof to Appendix B.2.

2.5 Modulus Switching and Key Switching

Lemma 3 (Modulus Switching). *Let $c = (b, a) \in R_q^2$ be an RLWE encryption of $m \in R_q$ with noise variance σ_{in}^2 . Assuming that $q' \mid q$, the modulus switching operation from q to q' is defined by:*

$$c' = \left(\left\lfloor \frac{q'}{q} b \right\rfloor, \left\lfloor \frac{q'}{q} a \right\rfloor \right) \in R_{q'}^2,$$

where $\lfloor \cdot \rfloor$ denotes coefficient-wise rounding. The resulting ciphertext c' encrypts the message $\frac{q'}{q} \cdot m$ with noise variance bounded by $\left(\frac{q'}{q}\right)^2 \sigma_{in}^2 + \sigma_{ms}^2$, where $\sigma_{ms}^2 \leq \frac{N+1}{12}$.

Lemma 4 (Key Switching). *Let $c = (b, a) \in R_q^2$ be an RLWE encryption of $m \in R_q$ with noise variance σ_{RLWE}^2 under secret s , and let C be an RLEV encryption of s with gadget parameters d, B and noise variance σ_{RLEV}^2 under s' . The key switching operation for c from secret s to s' is defined as:*

$$\text{KS}(c, C) = c' = (b, 0) + a \odot C \in R_q^2.$$

Assuming the RLWE ciphertext is uniformly distributed over R_q^2 , the resulting ciphertext c' satisfies $\varphi_{s'}(c') = m + e' \pmod{q}$ with noise variance bounded by $\sigma_{RLWE}^2 + \sigma_{ks}^2$ where

$$\sigma_{ks}^2 \leq dN \cdot \frac{B^2}{12} \cdot \sigma_{RLEV}^2 + N \cdot \frac{\epsilon^2}{3}$$

for some $\epsilon = \frac{q}{B^d}$.

The proofs of Lemmas 3 and 4 appear in Appendices B.3 and B.4, respectively.

3 HERDS

In this section, we present HERDS, a hierarchical multi-key LWE-based bootstrapping framework. Our key observation is that the scheme of Kwak et al. [33], compared to that of Chen et al. [10], requires executing blind rotation multiple times for each stack of RLEV ciphertexts and must employ a larger ring degree to control noise growth. Notably, this enlarged ring degree can also be exploited to support horizontal packing [16]. Motivated by this, we adopt a similar strategy in which a single blind rotation suffices to generate the desired RLEV ciphertexts.

At a high level, our hierarchical framework comprises two core components: (i) a first-layer ciphertext conversion that transforms a multi-key LWE ciphertext into k RLEV ciphertexts (Sec. 3.1); and (ii) a second-layer hybrid accumulator that aggregates these k RLEV ciphertexts into a single multi-key RLWE ciphertext (Sec. 3.2).¹⁰

3.1 Layer 1: Multi-key LWE to RLEV Ciphertext Conversion

Let $(b, \mathbf{a}_1, \dots, \mathbf{a}_k)$ denote a multi-key LWE ciphertext under the joint secret $\mathbf{z}_1, \dots, \mathbf{z}_k$, where for each $i \in [k]$, we have $\mathbf{a}_i = (a_{i,0}, \dots, a_{i,n-1})$ and $\mathbf{z}_i = (z_{i,0}, \dots, z_{i,n-1})$. In this section, we first show how to obtain k RLEV ciphertexts $\{C_i\}_{i \in [k]}$ with a power of two length d , where each C_i encrypts the monomial $X^{\sum_j \hat{a}_{i,j} z_{i,j}}$ for some $\hat{a}_{i,j} = \left\lfloor \frac{a_{i,j} \cdot 2N}{d \cdot q} \right\rfloor \cdot d \Big|_{2N}$. With a binary message space (as in [10,33]), the precision does not need to be large. So adding extra $\log d$ zeros in the LSB does not harm the precision. By doing so, we can evaluate several LUTs with a single bootstrapping, thereby reducing the overall time complexity of the blind rotation algorithm.

Step 1: MK-LWE to RLWE Blind Rotation from Block binary LWE.

In [34], Lee et al. introduced *block binary LWE* together with an optimized blind rotation for the TFHE framework, reducing the number of blind rotation iterations from n to n/ℓ for some integer ℓ and lowering noise throughout all phases of bootstrapping. We incorporate this optimization into our algorithm.

Definition 5 (Block Binary LWE). Let $q, n = \ell \cdot n'$ be positive integers, and let B_ℓ be a distribution over $\mathbb{Z}^\ell$, which samples one of the standard unit vectors in $\mathbb{Z}^\ell$ or the zero vector with equal probability. Let χ be a noise distribution over $\mathbb{Z}$. The Block Binary LWE problem, denoted by $\text{Block-LWE}_{n,q,\ell,\chi}$, asks an adversary $\mathcal{A}$ to distinguish between:

¹⁰ A complete description of HERDS also involves coefficient extraction and key-switching, similar to [10,33,59]. We omit these details here for clarity, focusing on the core components.

- $(b = \langle \mathbf{a}, \mathbf{z} \rangle + e \pmod{q}, \mathbf{a})$ where $\mathbf{a} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$, $\mathbf{z} = (\mathbf{z}_1, \dots, \mathbf{z}_{n'})$ with each $\mathbf{z}_i \leftarrow B_\ell$, and $e \leftarrow \chi$;
- $(v, \mathbf{u})$ where $\mathbf{u} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$, $v \leftarrow \mathcal{U}(\mathbb{Z}_q)$.

The Block-LWE $_{n,q,\ell,\chi}$ assumption states that any polynomial-time adversary $\mathcal{A}$ achieves a negligible advantage in distinguishing these distributions.

The Block-LWE $_{n,q,\ell,\chi}$ problem can be theoretically reduced to the standard LWE assumption, as its secret exhibits sufficiently high min-entropy under appropriate parameter settings [27]. For concrete instantiations, however, we follow the approach of [34] and rely on concrete security estimates based on practical attacks, rather than asymptotic reductions. For further technical details, we refer the reader to [34].

We now present the formal description of our construction for MK-LWE to RLWE blind rotation from block binary LWE. Given a multi-key LWE ciphertext $(b, \mathbf{a}_1, \dots, \mathbf{a}_k)$ under the joint secret $\bar{\mathbf{z}} = (\mathbf{z}_1, \dots, \mathbf{z}_k)$, the blind rotation key $\{\text{BRK}_i = \{\text{brk}_{i,j}\}_{j \in [0, n-1]}\}_{i \in [k]}$ where each $\text{brk}_{i,j}$ is an RGSW encryption of $z_{i,j}$ under the secret s_i , run Algorithm 1 as follows (we assume that d is power of two):

Algorithm 1 MK-LWE to RLWE blind rotation from block binary LWE

Input:

- The $\mathbf{a}$ -part $(\mathbf{a}_1, \dots, \mathbf{a}_k)$ of MK-LWE ciphertext
- The blind rotation key $\{\text{BRK}_i = \{\text{brk}_{i,j}\}_{j \in [0, n-1]}\}_{i \in [k]}$
- The gadget vector $\mathbf{g} = (g_0, \dots, g_{d-1}) \in \mathbb{Z}^d$

Output:

A set of RLWE ciphertext $\{(b_i, a_i) \in R_q^2\}_{i \in [k]}$.

- 1: **for** $i = 1$ to k **do**
 - 2: **for** $j = 0$ to $n - 1$ **do**
 - 3: $\hat{a}_{i,j} \leftarrow \left\lceil \left[\frac{a_{i,j} \cdot 2N}{d \cdot q} \right] \cdot d \right\rceil_{2N}$
 - 4: **for** $i = 1$ to k **do**
 - 5: $\text{ACC}_i \leftarrow (\sum_{j=0}^{d-1} g_j X^j, 0)$
 - 6: **for** $j = 0$ to $n/\ell - 1$ **do**
 - 7: $\text{dACC}_i \leftarrow \mathbf{g}^{-1}(\text{ACC}_i)$
 - 8: $\text{ACC}_i \leftarrow \text{ACC}_i + \sum_{w=0}^{\ell-1} (X^{\hat{a}_{i,j\ell+w}} - 1) \cdot \text{dACC}_i^\top \cdot \text{brk}_{i,j\ell+w} \pmod{q}$
 - 9: **return** $\text{ACC} \leftarrow (\text{ACC}_1, \dots, \text{ACC}_k)$
-

Lemma 5. Let $(b, \{\mathbf{a}_i\}_{i \in [k]})$ be an MK-LWE ciphertext under the joint secret $\bar{\mathbf{z}} = (\mathbf{z}_1, \dots, \mathbf{z}_k) \in \{0, 1\}^{kn}$. Assume $\bar{\mathbf{z}}$ follows the block-binary distribution with block length ℓ . For each $i \in [k]$ and $w \in [0, n-1]$, define the scaled coefficient $\hat{a}_{i,w} \leftarrow \left\lceil \left[\frac{a_{i,w} \cdot 2N}{d \cdot q} \right] \cdot d \right\rceil_{2N}$. Moreover, for all $i \in [k]$ and $j \in [n]$, let $\text{brk}_{i,j}$ be an RGSW ciphertext of $z_{i,j}$ with gadget parameters d_{br} and B_{br} and noise variance σ_{brk}^2 under secret s_i . Then, for the output $\{\text{ACC}_i\}_{i \in [k]}$ of Algorithm 1, each ACC_i

is an RLWE ciphertext satisfying

$$\varphi_{s_i}(\text{ACC}_i) = e_i + \sum_{j=0}^{d-1} g_j X^{j+\sum_{w=0}^{n-1} \hat{a}_{i,w} z_{i,w}} \pmod{q},$$

for some small e_i whose variance σ_{br}^2 is bounded by

$$\sigma_{br}^2 \leq \frac{1}{3} d_{br} n N B_{br}^2 \sigma_{brk}^2 + \frac{2n}{3\ell} (N+1) \epsilon_{br}^2, \quad \epsilon_{br} = \frac{q}{B_{br}^{d_{br}}}.$$

See Appendix B.5 for the proof of Lemma 5.

Step 2: Homomorphic Decode. After Step 1, we obtain k accumulators ACC_i for $1 \leq i \leq k$, where each $\text{ACC}_i = (b_i, a_i)$ is an RLWE ciphertext with phase

$$\varphi_{s_i}(\text{ACC}_i) = e_i + \sum_{j=0}^{d-1} g_j X^{j+\sum_{w=0}^{n-1} \hat{a}_{i,w} z_{i,w}} \pmod{q}.$$

In this step, we apply the *homomorphic decode* algorithm to each ACC_i , which transforms it into a valid RLEV ciphertext C_i that encrypts the monomial $X^{\sum_{w=0}^{n-1} \hat{a}_{i,w} z_{i,w}}$. To control noise growth during the homomorphic decode, we first preprocess each ACC_i as [58].

- **NTT-Friendly Modulus (Prime Q).** In this setting, d is invertible modulo q , and we can directly compute the scaled accumulator by multiplying each ACC_i with $d^{-1} \pmod{q}$. The resulting ciphertext ACC'_i has phase

$$\varphi_{s_i}(\text{ACC}'_i) = \frac{1}{d} \sum_{j=0}^{d-1} g_j X^{j+\sum_{w=0}^{n-1} \hat{a}_{i,w} z_{i,w}} + \frac{1}{d} e_i \pmod{q}.$$

- **FFT-Friendly Modulus (Power-of-Two Q).** When d is not invertible modulo q , we perform a two-step procedure:
 1. *Modulus Switching:* Switch modulus from q to $q' = q/d$, yielding an intermediate ciphertext with phase

$$\frac{1}{d} \sum_{j=0}^{d-1} g_j X^{j+\sum_{w=0}^{n-1} \hat{a}_{i,w} z_{i,w}} + \frac{1}{d} e_i + e_{ms,i} \pmod{q'},$$

where $e_{ms,i}$ denotes the modulus switching noise.

2. *Modulus Raising:* Lift the ciphertext back to modulus q via modulus raising, resulting in a final ciphertext with phase

$$\varphi_{s_i}(\text{ACC}'_i) = \frac{1}{d} \left(\sum_{j=0}^{d-1} g_j X^{j+\sum_{w=0}^{n-1} \hat{a}_{i,w} z_{i,w}} + e_i \right) + e_{ms,i} + q' \cdot \zeta_i \pmod{q},$$

where $\zeta_i \in R$ is an integer coefficient polynomial arising from the modulus raising.

Since we set $q = 2^{64}$ in our implementation, our setting corresponds to the latter case. Let $\{\text{TRK}_i = \{\text{trk}_{i,j}\}_{j \in [0, \log_2 d - 1]}\}_{i \in [k]}$ denote the collection of trace keys, where each $\text{trk}_{i,j}$ is an RLEV encryption of $s_i(X^{\frac{N}{2^j}+1})$ under the secret key $s_i(X)$. After preprocessing, the k RLWE ciphertexts $\{\text{ACC}'_i = c'_i = (b'_i, a'_i)\}_{i \in [k]}$ must be converted into k RLEV ciphertexts. To this end, we introduce two homomorphic decoding algorithms: an efficiency-oriented variant (Algorithm 2) and a parallelism-oriented variant (Algorithm 3).

Algorithm 2 Homomorphic Decode (Efficiency-oriented)

Input:

Notation: $[\cdot]$ denotes an array, $\ell = \log_2 d$ for some power-of-two d

k RLWE ciphertexts $\{c'_i = (b'_i, a'_i)\}_{i \in [k]}$

Trace-key sets $\{\text{TRK}_i = \{\text{trk}_{i,w}\}_{w \in [0, \ell - 1]}\}_{i \in [k]}$

Output:

k RLEV ciphertexts $\{C_i \in R_q^{d \times 2}\}_{i \in [k]}$

```

1: for  $i = 1$  to  $k$  do
2:    $c_i^{(0)} \leftarrow c'_i$ 
3:    $\text{arr}_i^{(0)} \leftarrow [c_i^{(0)}]$ 
4:   for  $w = 0$  to  $\ell - 1$  do
5:      $\text{arr}_i^{(w+1)} \leftarrow []$ 
6:      $t \leftarrow N/2^w + 1$ 
7:     for  $j = 0$  to  $|\text{arr}_i^{(w)}| - 1$  do
8:        $c_0 \leftarrow \text{arr}_i^{(w)}[j]$ 
9:        $c_1 \leftarrow c_0 \cdot X^{-2^w}$ 
10:       $\tilde{c}_0 \leftarrow \text{KS}(\phi_t(c_0), \text{trk}_{i,w})$ 
11:       $\tilde{c}_1 \leftarrow \text{KS}(\phi_t(c_1), \text{trk}_{i,w})$ 
12:       $c_j^{(w+1)} \leftarrow c_0 + \tilde{c}_0$ 
13:       $c_{j+2^w}^{(w+1)} \leftarrow c_1 + \tilde{c}_1$ 
14:      Append  $c_j^{(w+1)}$  and  $c_{j+2^w}^{(w+1)}$  to  $\text{arr}_i^{(w+1)}$ 
15:    $C_i \leftarrow \text{arr}_i^{(\ell)}$ 
16: return  $\{C_1, \dots, C_k\}$ 

```

Lemma 6. Assume each c'_i is an RLWE ciphertext with phase $\varphi_{s_i}(c'_i)$, where

$$\varphi_{s_i}(c'_i) = \frac{1}{d} \left(\sum_{j=0}^{d-1} g_j X^{j + \sum_{w=0}^{n-1} \hat{a}_{i,w} z_{i,w}} \right) + e'_i \bmod q, \quad e'_i = \frac{1}{d} e_i + e_{ms,i} + \frac{q}{d} \zeta_i.$$

Moreover, the variance bounds of e_i and $e_{ms,i}$ are σ_{br}^2 and σ_{ms}^2 , respectively. Let $\{\text{TRK}_i = \{\text{trk}_{i,j}\}_{j \in [0, \lceil \log_2 d \rceil - 1]}\}_{i \in [k]}$ be the trace keys where each $\text{trk}_{i,j}$ is an RLEV encryption of $s_i(X^{\frac{N}{2^j}+1})$ under the secret $s_i(X)$ with gadget parameters d_{tr} and B_{tr} and a noise variance σ_{trk}^2 . Then, for the output $\{C_i\}_{i \in [k]}$ by Algorithm 2 and Algorithm 3, each C_i is an RLEV ciphertext satisfying

$$\varphi_{s_i}(C_i) = X^{\sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}} + e''_i \bmod q$$

Algorithm 3 Homomorphic Decode (Parallelism-oriented)

Input:

k RLWE ciphertext $\{c'_i = (b'_i, a'_i)\}_{i \in [k]}$

The trace key set $\{\text{TRK}_i = \{\text{trk}_{i,j}\}_{j \in [0, \log_2 d - 1]}\}_{i \in [k]}$

Output:

k RLEV ciphertext $\{C_i \in R_q^{d \times 2}\}_{i \in [k]}$

1: **for** $i = 1$ to k **do**

2: **for** $j = 0$ to $d - 1$ **do**

3: $c_{i,j} \leftarrow X^{-j} \cdot c_i$

4: **for** $w = 0$ to $\log_2 d - 1$ **do**

5: $c_{i,j} \leftarrow c_{i,j} + \text{KS}\left(\phi_{\frac{N}{2^w}+1}(c_{i,j}), \text{trk}_{i,w}\right)$

6: **return** $(C_1, \dots, C_k) \leftarrow \left(\begin{bmatrix} c_{1,0} \\ \vdots \\ c_{1,d-1} \end{bmatrix}, \dots, \begin{bmatrix} c_{k,0} \\ \vdots \\ c_{k,d-1} \end{bmatrix} \right)$

for some small e''_i , whose variance is bounded by

$$\sigma_{br}^2 + d^2 \sigma_{ms}^2 + \frac{d^2 - 1}{3} \cdot \sigma_{ks}^2,$$

where $\sigma_{br}^2 \leq \frac{1}{3} d_{br} n N B_{br}^2 \sigma_{brk}^2 + \frac{2n}{3\ell} (N+1) \epsilon_{br}^2$, $\sigma_{ms}^2 \leq \frac{N+1}{12}$, and $\sigma_{ks}^2 \leq d_{tr} N \cdot \frac{B_{tr}^2}{12} \cdot \sigma_{trk}^2 + N \cdot \frac{\epsilon_{tr}^2}{3}$ respectively, for some $\epsilon_{br} = q/B_{br}^{d_{br}}$ and $\epsilon_{tr} = q/B_{tr}^{d_{tr}}$.

See Appendix B.6 for the proof of Lemma 6.

3.2 Layer 2: RLEV to Multi-key RLWE Accumulation

Consider a multi-key LWE ciphertext $(b, \mathbf{a}_1, \dots, \mathbf{a}_k)$ under secrets $\mathbf{z}_1, \dots, \mathbf{z}_k$, where $\mathbf{a}_i = (a_{i,0}, \dots, a_{i,n-1})$ and $\mathbf{z}_i = (z_{i,0}, \dots, z_{i,n-1})$ for each $i \in [k]$. After the first layer, we obtain k RLEV ciphertexts $\{C_i\}_{i \in [k]}$, each encrypting $X^{\sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}}$ under secret key s_i , with noise variance bounded by $\sigma_{br}^2 + d^2 \sigma_{ms}^2 + \frac{d^2-1}{3} \sigma_{ks}^2$. The objective of this step is to aggregate these ciphertexts into a multi-key RLWE ciphertext under the joint secret $\bar{s} = (s_1, \dots, s_k)$, which encrypts the message $r(X) \cdot X^{\hat{b} + \sum_{i=1}^k \sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}}$, where $\hat{b} = \lceil [b \cdot 2N / (dq)] \cdot d \rceil_{2N}$. This aggregation is realized via a hybrid product between RLEV and MK-RLWE ciphertexts with respect to the test vector $r(X)$. Formally, let $\mathbf{pp}_r = (N, \psi', \chi', d_r, B_r, \mathbf{a}'_1, \mathbf{a}'_2)$ denote the public parameters, where N is the ring dimension, ψ' and χ' are the secret-key and noise distributions over R_q , d_r and B_r are the gadget decomposition parameters, $\mathbf{g}_r$ and $\mathbf{g}_r^{-1}$ are the corresponding gadget vector and decomposition function, and $\mathbf{a}'_1, \mathbf{a}'_2 \in R^{d_r}$ are common random strings. The hybrid product is then defined by a pair of polynomial-time algorithms (V2M.KGen, V2M.Eval).

- V2M.KGen($s_i, \mathbf{pp}_r$): Each party i independently generates its public key $\mathbf{pk}_i$ and relinearization key $\mathbf{rlk}_i$ as follows:

- Sample noise $\mathbf{e}_{i,0} \leftarrow \chi'^{d_r}$ and compute the public key:

$$\mathbf{pk}_i = -\mathbf{a}'_1 \cdot s_i + \mathbf{e}_{i,0} \in R_q^{d_r}$$

For simplicity, we set $s_0 = 1$ and $\mathbf{pk}_0 = -\mathbf{a}'_1$.

- Sample $r_i \leftarrow \psi'$ and noise $\mathbf{e}_{i,1}, \mathbf{e}_{i,2} \leftarrow \chi'^{d_r}$, then compute:

$$\mathbf{d}_i = \mathbf{a}'_1 \cdot r_i + \mathbf{g}_r \cdot s_i + \mathbf{e}_{i,1}, \quad \mathbf{f}_i = -\mathbf{a}'_2 \cdot s_i + \mathbf{g}_r \cdot r_i + \mathbf{e}_{i,2} \quad (\in R_q^{d_r})$$

The relinearization key is defined as $\mathbf{rlk}_i = (\mathbf{d}_i, \mathbf{f}_i) \in R_q^d \times R_q^{d_r}$.

- **V2M.Eval**($\{\mathbf{rlk}_i, \mathbf{pk}_i\}_{i \in [k]}, \{C_i\}_{i \in [k]}, r(X)X^{\hat{b}}$): Given RLEV ciphertexts C_i of $X^{\sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}}$ under the secret key s_i for $1 \leq i \leq k$ along with their associated relinearization keys and public keys $\{\mathbf{rlk}_i, \mathbf{pk}_i\}_{i \in [k]}$, and the test vector $r(X)X^{\hat{b}}$, Algorithm 4 computes and outputs a multi-key RLWE ciphertext $\bar{c} = (c_0, c_1, \dots, c_k) \in R_q^{k+1}$.

Algorithm 4 RLEV-MK-RLWE Hybrid Accumulation

Input:

The RLEV ciphertexts $\{C_i\}_{i \in [k]}$

The public keys and relinearization keys $\{\mathbf{rlk}_i, \mathbf{pk}_i\}_{i \in [k]}$

The test vector $r(X)X^{\hat{b}}$

Output:

A multi-key RLWE ciphertext $\bar{c} = (c_0, c_1, \dots, c_k) \in R_q^{k+1}$

- 1: $(c_0, \dots, c_k) \leftarrow (r(X)X^{\hat{b}}, 0, 0, \dots, 0)$
 - 2: **for** $i = 1$ to k **do**
 - 3: **for** $j = 0$ to $i - 1$ **do**
 - 4: $(w_j, u_j) \leftarrow c_j \odot C_i$
 - 5: $c_j \leftarrow \langle \mathbf{g}_r^{-1}(u_j), \mathbf{d}_i \rangle + w_j$
 - 6: $v_j \leftarrow \langle \mathbf{g}_r^{-1}(u_j), \mathbf{pk}_j \rangle$
 - 7: $v \leftarrow \sum_{j=0}^{i-1} v_j$
 - 8: $c_0 \leftarrow c_0 + \langle \mathbf{g}_r^{-1}(v), \mathbf{f}_i \rangle$
 - 9: $c_i \leftarrow c_i + \langle \mathbf{g}_r^{-1}(v), \mathbf{a}'_2 \rangle$
 - 10: **return** $(c_0, \dots, c_k)$
-

Lemma 7. Assume each C_i is an RLEV ciphertext that encrypts $X^{\sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}}$ and the variance of its noise is bounded by $\sigma_{br}^2 + d^2 \sigma_{ms}^2 + \frac{d^2-1}{3} \sigma_{ks}^2$. Let $\{\mathbf{rlk}_i, \mathbf{pk}_i\}_{i \in [k]}$ be the relinearization keys and public keys defined as above with gadget parameters d_r and B_r and a noise variance σ_r^2 . Then, for some test vector $r(X)X^{\hat{b}}$, the output $\bar{c} = (c_0, c_1, \dots, c_k)$ of Algorithm 3 is an MK-RLWE ciphertext satisfying

$$\varphi_{\bar{s}}(\bar{c}) = r(X) \cdot X^{\hat{b} + \sum_{i=1}^k \sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}} + e \mod q$$

for some small e , whose variance is approximately bounded by

$$\left(\frac{k(k-1)}{2} N + k \right) \cdot \left(\frac{dNB^2}{12} \left(\sigma_{br}^2 + d^2 \sigma_{ms}^2 + \frac{d^2-1}{3} \sigma_{ks}^2 \right) + \frac{\epsilon^2}{3} \right) + (k^2 N + k) \frac{d_r NB_r^2}{12} \sigma_r^2$$

where $\epsilon = \frac{q}{B^d}$.

We defer the proof to Appendix B.7.

3.3 Making HERDS More Parallel-Friendly

Although we’ve reduced the complexity of bootstrapping from $O(dkn)$ to $O(n)$, a critical bottleneck persists: the core single-key blind rotation operation still requires $O(n)$ sequential steps, which severely hinders parallelization. In this section, we break this serial dependency by redesigning the underlying data representation. Our objective is to exploit the fundamental structural independence inherent at the ciphertext level across our two-layer computational process, making the scheme particularly amenable to acceleration on modern multi-core architectures, GPUs, and specialized hardware platforms.

Our core innovation is a tensor-based reorganization of the vector space. For each vector $\mathbf{a}_i = (a_{i,0}, \dots, a_{i,n-1})$ in the MK-LWE ciphertext, we establish an isomorphism by factoring the dimension as $n = \omega_1 \cdot \omega_2$. This enables us to represent the vector as a matrix, facilitating a decomposition into ω_2 independent blocks:

$$\mathbf{a}_i = \mathbf{S}_{i,1} \parallel \mathbf{S}_{i,2} \parallel \dots \parallel \mathbf{S}_{i,\omega_2},$$

where each block $\mathbf{S}_{i,\ell} \in \mathbb{Z}^{\omega_1}$ is a sub-vector of length ω_1 . This decomposition enables a highly parallel computational framework with two distinct levels of parallelism, which we now describe.

Intra-party Parallelism in Layer 1. By converting the one-dimensional vector of length n into a two-dimensional vector with ω_2 blocks, each of length ω_1 , the MK-LWE-to-RLEV ciphertext conversion (Step 1) described in Algorithm 1 can be applied independently to each block. Notably, during the RLWE-to-RLEV ciphertext conversion process, we show that each RLEV polynomial can be separately generated with a parallelism degree d using homomorphic decoding (Step 2) as presented in Algorithm 3. This allows for the full parallel execution of Lines 3-5 in Algorithm 3. Based on the above analysis of intra-party parallelism, a single party can independently generate ω_2 RLEV ciphertexts.

Inter-party Parallelism in Layer 1. In Layer 1, the computations among the k parties are completely independent, with no data dependencies between them. This characteristic allows all parties to perform *intra-party* computations concurrently, enabling them to simultaneously generate $k\omega_2$ RLWE ciphertexts in Step 1 and $k\omega_2$ RLEV ciphertexts in Step 2.

Intra-party Parallelism in Layer 2. In Layer 2, this data independence between parties is also evident in the RLEV-to-MKRLWE conversion process. Specifically, Lines 4-6 in Algorithm 4 can be executed in parallel to generate c_j and v_j , which are then used for subsequent computations.

3.4 Analysis and Comparison

In this section, we analyze the computational complexity of HERDS’s bootstrapping algorithm in both serial and parallel settings.

Theorem 1. *For k parties and lattice dimension n , HERDS achieves bootstrapping under the same polynomially large modulus $q \in \tilde{O}(kn^2)$ with serial complexity $O(kn)$ and parallel complexity $O(\sqrt{nk})$.*

Proof. We begin by analyzing the noise growth in HELIOS. From Lemma 3–7, the variance of the final output noise is bounded by $\tilde{O}(k^2n^4)$. Hence, the standard deviation is $\tilde{O}(kn^2)$. By adopting the conventional bound of six times the standard deviation as the noise upper bound, and requiring this quantity to remain below $q/16$ (as in [16]), it follows that $q \in \tilde{O}(kn^2)$ is sufficient.

We now analyze the computational complexity using the gadget product $\langle \mathbf{g}^{-1}(\cdot), \cdot \rangle$ as the fundamental cost unit, assuming uniform gadget parameters across all operations. Each gadget product requires d ring multiplications in $\mathcal{R}_q$. Algorithm 1 executes k blind rotations, each consisting of n/ℓ RLWE–RGSW external products, resulting in $4kn/\ell$ gadget products ($\ell = 2$ in our setting). Algorithm 2 (resp. Algorithm 3) converts k RLWE ciphertexts into k RLEV ciphertexts via kd (resp. $kd \log d$) key-switching operations at 2 gadget products each, totaling $2kd$ (resp. $2kd \log d$). Since $d = O(1)$ and typically $d \ll n$, this term is asymptotically negligible. Algorithm 4 performs hybrid multiplications between k RLEV ciphertexts and one multi-key RLWE ciphertext, requiring $\sum_{i=1}^k (4(i-1)+2) = 2k^2$ gadget products. The overall computational complexity is $O(kn + k^2)$, which reduces to $O(kn)$ under the typical regime $k \ll n$.

We now formally characterize the efficiency and noise growth of our scheme through the following theorem.

Theorem 2. *For k parties and lattice dimension n , HELIOS achieves bootstrapping with parallel computational complexity $O(k\sqrt{n})$ and noise growth bounded by $O(\sqrt{kn})$ under a polynomially large modulus.*

Proof. We begin by analyzing the noise growth in the parallel-friendly workflow described above. We show that it does not incur additional asymptotic noise compared to the sequential approach. By Lemma 7, the dominant noise term is $\frac{k(k-1)}{2} \cdot \frac{dN^2B^2}{12} \sigma_{br}^2$, where one factor of k comes from the size of the multi-key RLWE accumulator in the second layer, and the other from the number of RLEV ciphertexts generated in the first layer. In our construction, the number of RLEV ciphertexts increases from k to $k\omega_2$, adding a factor of ω_2 to the total noise. At the same time, the number of blind rotation steps is reduced from $\frac{n}{l}$ to $\frac{n}{\omega_2 l}$, reducing σ_{br}^2 by the same factor. These two effects cancel out, and the overall noise bound remains asymptotically the same.

We now analyze the algorithm’s complexity under the two-layer parallelization strategy. Each of the ω_2 blind rotations requires $O(\omega_1)$ operations, while k parties compute in parallel. Subsequent homomorphic decoding involves $\log d$ key-switching operations (parallelizable), and homomorphic accumulation over $k\omega_2$ RLEV ciphertexts incurs $O(k\omega_2)$ cost. Thus, the parallel complexity is $O(\omega_1 + \log d + k\omega_2)$. With $d = O(1)$ and $n = \omega_1\omega_2$, this reduces to $O\left(\frac{n}{\omega_2} + k\omega_2\right)$. Balancing the terms gives $\omega_2 = \sqrt{n/k}$ and $\omega_1 = \sqrt{nk}$, achieving an optimal parallel complexity of $O(\sqrt{nk})$. $\square$

In Table 1, we compare the theoretical complexities of our scheme with prior MK-TFHE constructions. Our scheme achieves the best asymptotic performance, with the parallel version attaining sublinear growth in both n and k . Fig. 3, based on the experiments in Sec. 3.4, further shows that the bootstrapping runtime scales consistently with our theoretical analysis under both serial (CPU) and parallel (GPU) settings

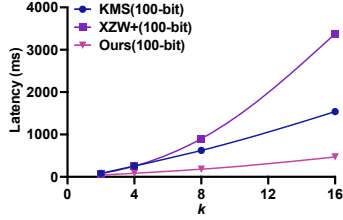


Fig. 1. *
(a) Serial mode

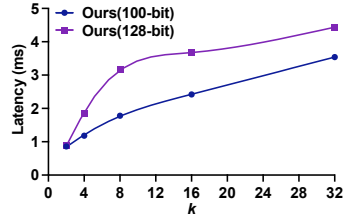


Fig. 2. *
(b) Parallel mode

Fig. 3. Bootstrapping runtime vs. number of parties k under (a) serial and (b) parallel settings.

4 Distributed Decryption for Multi-Key Ciphertexts

In this section, we propose a new distributed decryption protocol which does not require a cumbersome bootstrapping under a large parameter. In our protocol, an interactive bootstrapping is performed to obtain a ciphertext with a small noise bound without a need for large parameters. Briefly speaking, the parties can collaboratively simulate blind rotation by physically rotating predetermined LWE ciphertexts sequentially, which is performed with a compact noise growth of the size of only a public key encryption noise. As a result, the ciphertext modulus can be compact enough to accommodate just the flooding and public key encryption noise.

4.1 Multi-key PK Encryption

We begin by describing the public-key encryption scheme based on the (R)LWE assumption, under a multi-key setting.

- **Setup(1^λ)** : Given the security parameter λ , choose the ring degree N , key and noise distributions ψ' and χ' over R . Output the public parameters $\text{pp} = (N, \psi', \chi')$.
- **Keygen(pp)** : Each i -party does the following:

- Samples $\mathbf{pk}_{i,1} \leftarrow \mathcal{U}(R_q)$, $s_i \leftarrow \psi'$, and $e_i \leftarrow \chi'$. Sets $\mathbf{pk}_{i,0} = -\mathbf{pk}_{i,1} \cdot s_i + e_i \pmod{q}$ and discloses $\mathbf{pk}_i = (\mathbf{pk}_{i,0}, \mathbf{pk}_{i,1}) \in R_q^2$.
- Upon receiving $\{\mathbf{pk}_i\}_{1 \leq i \leq k}$, compute $\mathbf{pk} = (\sum_{i=1}^k \mathbf{pk}_{i,0}, \mathbf{pk}_{1,1}, \dots, \mathbf{pk}_{k,1}) \in R_q^{k+1}$.

• **EncZero(pk)** : Does the following:

- Samples $\gamma \leftarrow \psi'$ and $e_i \leftarrow \chi'$ for $0 \leq i \leq k$, and computes $(c_0, \dots, c_k) = \gamma \cdot \mathbf{pk} + (e_0, \dots, e_k) \pmod{q}$.
- Parse $c_i = \sum_{j=0}^{N-1} c_{i,j} X^j$, and outputs

$$(c_{0,0}, c_{1,0}, -c_{1,N-1}, \dots, -c_{1,1}, c_{2,0}, -c_{2,N-1}, \dots, -c_{2,1}, \dots, -c_{k,1}) \in \mathbb{Z}_q^{kN+1}.$$

Assuming that the parties followed the key generation protocol honestly, the output distribution of the multi-key zero encryption generation algorithm below is computationally indistinguishable from the uniform distribution over $\mathbb{Z}_q^{kN+1}$, assuming that $\text{RLWE}_{N,q,\psi',\chi'}$ holds. In practice, we choose ψ' to be a binary/ternary distribution, and χ' a Gaussian distribution with a small width, which guarantees a small noise bound of $O(N)$.

4.2 Protocol Description

Based on the multi-key public key encryption scheme from the previous section, we provide the complete protocol in Fig. 4. In the description, D_{flood} denotes some exponentially large distribution, which can flood the public key encryption noise. We remark that the algorithm is valid for $q' \geq \frac{4}{3}(k+1)$ if the input ciphertext noise is bounded by $q/16$, as the rounding noise is bounded by $\frac{k+1}{2}$.

It is worth noting that this distributed decryption protocol can be extended to arbitrary plaintext BFV ciphertexts. Also, the baseline public key encryption scheme does not need to be LWE, but any public key encryption scheme which can hide the rotation index. The underlying public key scheme can be any threshold/multi-key linearly homomorphic scheme that can successfully hide the rotation index with a public key zero encryption. For example, threshold Paillier [24], or LWE-based public key encryptions [46,50] can be utilized, potentially without the need of the noise flooding technique. However, these works may require a trusted key dealer, or a trusted server and secure channel.

Theorem 3. *The distributed decryption protocol Π_{DDec} (Fig. 4) securely implements $\mathcal{F}_{\text{DDec}}$ (Fig. 5) in the presence of any semi-honest, passive adversary corrupting at most $k-1$ parties.*

4.3 Proof of Theorem 3

We provide a security proof of our distributed decryption protocol, given in Fig. 4.

Proof. For simplicity, assume that the adversary corrupts $k-1$ parties, where h is the index of the honest party. Moreover, we assume $h \neq k$, as extending our proof to the $h = k$ case is straightforward.

We build the simulator for the view of the adversary as follows.

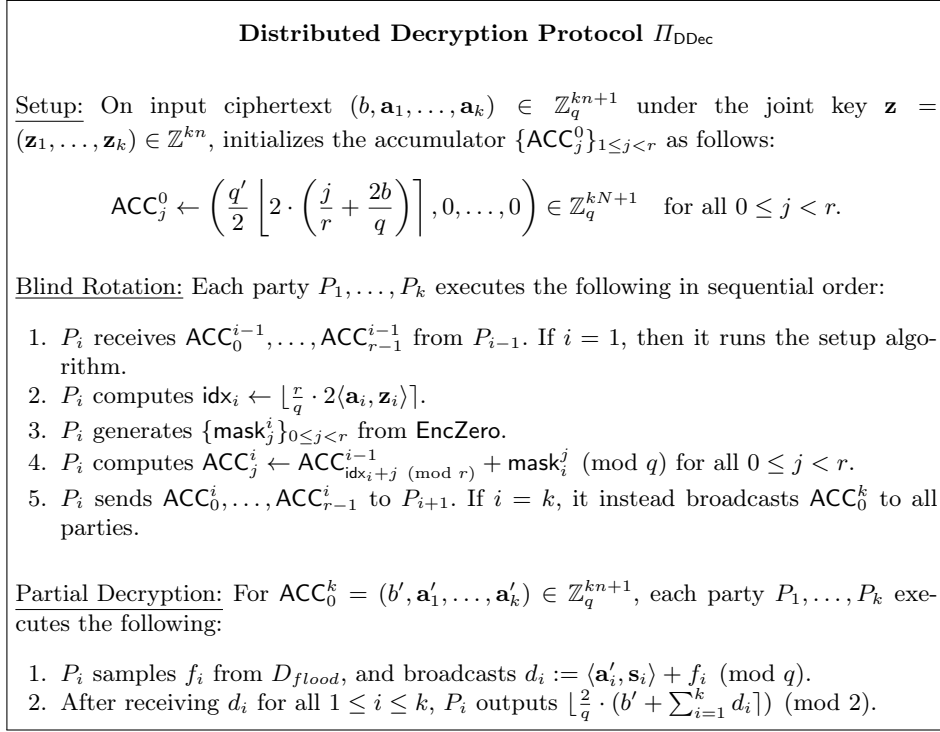


Fig. 4. Our distributed decryption protocol.

1. The simulator simulates $\text{ACC}_0^h, \dots, \text{ACC}_{r-1}^h$ by sampling them uniformly at random from $\mathbb{Z}_q^{kN+1}$.
2. The simulator computes $\text{ACC}_0^k = (b', \mathbf{a}'_1, \dots, \mathbf{a}'_k)$ by following the behavior of $P_{h+1}, \dots, P_k$ honestly.
3. Finally, it samples d_i for $i \neq h$ honestly and sets $d_h = \lfloor \frac{q}{4} \mu \rfloor - \frac{1}{8} + f_h - \sum_{i \neq h} d_i \pmod{q}$ where $f_h \leftarrow \mathcal{D}_{\text{flood}}$.

We proceed by the hybrid argument, starting from the real world.

Hyb₁. In this hybrid, we set $\text{ACC}_j^0 = \mathbf{0}$ for all $0 \leq j < r$, and set $d_h = b' - \sum_{i \neq h} \langle \mathbf{a}'_i, \mathbf{s}_i \rangle + (\lfloor \frac{q}{4} \mu \rfloor - \frac{q}{8}) + f_h$ where $f_h \leftarrow \mathcal{D}_{\text{flood}}$. The adversary's view in **Hyb₁** is statistically indistinguishable from the real world by the smudging lemma. The proof is standard and we refer the reader to [4, Theorem 5.2].

Hyb₂. In this Hybrid, we replace ACC_j^h with a uniformly random vector over $\mathbb{Z}_q^{kN+1}$. Since

$$\gamma \cdot \mathbf{pk} + (e_0, \dots, e_k) = \left(\sum_{i=1}^k (-(\mathbf{pk}_{i,1} \gamma + e_i) s_i + \gamma \hat{e}_i + e_i s_i) + e_0, \mathbf{pk}_{1,1} \gamma + e_1, \dots, \mathbf{pk}_{k,1} \gamma + e_k \right),$$

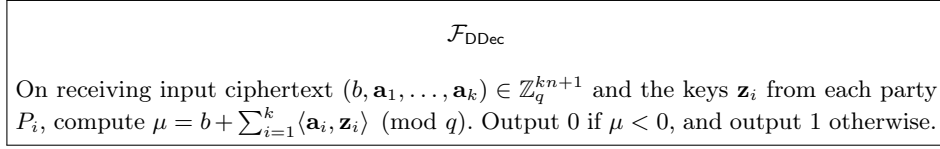


Fig. 5. The ideal functionality for distributed decryption protocol.

the distribution of `mask` is computationally indistinguishable from $\mathcal{U}(\mathbb{Z}_q^{kN+1})$ by the RLWE assumption as the adversary does not know γ and s_h . Therefore, the adversary's view in the real world and hyb_1 is computationally indistinguishable.

Since **Hyb**₂ is identical to our simulator, we conclude that the simulator correctly simulates the view of the adversary. $\square$

4.4 Comparison

Finally, we compare the parameters of our distributed decryption protocol with Noah's Ark [20], which is the current state-of-the-art distributed decryption protocol for threshold TFHE. Roughly speaking, Noah's Ark takes the input ciphertext and bootstraps it with a large parameter, then finally leverages the noise flooding technique to decrypt the ciphertext. To allow this, the parties need to generate an additional bootstrapping key under a large parameter, so that the bootstrapped ciphertext can accommodate an exponential error even after the bootstrapping.

We firstly compare the parameter settings. In our protocol, it is clear that the only source of noise growth is adding `mask`. Since `mask` is simply a public key encryption of zero, the variance of its noise is bounded by $(2kN + 1)\sigma'^2$, assuming $\psi' \equiv \mathcal{U}(R_2)$ and $\chi' \equiv \mathcal{D}_{\sigma'}$. Since each party adds one `mask` to each accumulator, the variance of the noise of ACC after the interactive blind rotation phase is bounded by $k(2kN + 1)\sigma'^2$. Let us consider the case where $k = 16$. Then, with ring degree of $N = 2048$, we have $\sqrt{k(2kN + 1)\sigma'^2} \approx 2^{11.14}$. Since the maximum possible standard deviation for the noise with decryption failure rate less than 2^{-64} is approximately $2^{58.80}$, we can set $D_{flood} \equiv \mathcal{D}_{\sigma_{flood}}$ with $\sigma_{flood} \approx 1/k \cdot 2^{58.80} \approx 2^{53.80}$. This amounts to statistical security of $54.80 - 11.15 \approx 43$ bits. On the other hand, Noah's ark requires a larger parameter of $N = 4096$ with 128 bits of modulus, achieving 40 bits of statistical security, similar to ours. This is mainly due to the blind rotation noise, which consumes almost 80 bits of modulus. Also, it is worth noting that Noah's ark is based on the threshold FHE scheme which has the same parameter as a single-key scheme; therefore, applying it to a multi-key scheme must require even larger parameters than this.

In terms of timing, our protocol is light since it only consists of $k \cdot q' = O(k^2)$ public-key generations and communications. On the contrary, the running time of Noah's ark depends on the bootstrapping under the large parameters. However, this bootstrapping can be costly, as it needs to use $q = 2^{128}$ during decryption, which uses an expensive quadruple-precision floating-point implementation.

Moreover, with a larger ring degree of $N = 4096$, the timing can be even slower. In contrast, our distributed decryption protocol can use $q = 2^{64}$ as usual in the original scheme, allowing for much better performance.

5 Evaluation

5.1 Experimental Setup

Test Environments We conduct experiments on a MacBook Pro equipped with an Apple M4 Pro chip (12-core CPU, comprising 8 performance cores and 4 efficiency cores), 24 GB of unified memory, and running macOS Sequoia 15.4. All GPU-based experiments are conducted on a server running Ubuntu 20.04, equipped with an Intel (R) Xeon (R) W5-3435X CPU, 128 GB of system memory, and an NVIDIA GeForce RTX 4090 GPU with 24 GB of device memory. The implementation is developed using CUDA 12.3 and compiled with GCC 10.5.0 and CMake 3.26.4. Our implementation is publicly available at <https://github.com/SNUCP/mktfhe-parallel-go>

Parameter Sets To achieve a fair comparison, we also adopted the heuristic parameter-selection strategy used in prior works [1,33,10], i.e., independently running more than 1,000 decryption trials without observing any errors. We note that this is already better than the best RLWE-based method [33] (as reported in [1], Table 2). We present our recommended parameter sets in Table 3. Sets I–V support 2, 4, 8, 16, and 32 parties with at least 100-bit security, while sets I'–V' achieve 128-bit security for the same configurations (estimated via [2]). And [34] considered three attacks, i.e., (1) dual hybrid attack for sparse keys, (2) Meet-in-the-Middle (MitM) attack, (3) a hybrid of MitM and dual attacks. We have already considered all these attacks to choose the parameters. Specifically, for the (MK)LWE ciphertexts, we set the dimension $n = 560$ (resp. 636) and the noise standard deviation $\alpha = 3.05 \cdot 10^{-5}$ (resp. $6.05 \cdot 10^{-5}$) to achieve 100-bit (resp. 128-bit) security. For the bootstrapping parameters, we fix the ring degree $N = 2048$, with a uniform binary secret key and a Gaussian noise distribution of standard deviation $\beta = 1.2 \cdot 10^{-19}$ (resp. $4.4 \cdot 10^{-16}$) to support high-precision torus arithmetic (64-bit precision), in line with the target security levels. Furthermore, to optimize performance, we employ tailored gadget decomposition parameters for different components: the blind rotation key (BRK) in Algorithm 1, the trace key (TRK) in Algorithm 3, and both the public key (pk) and relinearization key (rlk) in Algorithm 4. For the complete parameter settings, one can refer to the full version for more details.

Benchmarking For a fair comparison, we implemented both HERDS and the state-of-the-art RLWE-based baseline using the TFHE-go library [28], and we additionally benchmarked against the NTRU-based bootstrapping algorithm available in OpenFHE [5]. All experiments were executed on the same platform under identical configurations.

Table 3. Recommended Parameter Sets with Security Levels.

Set	LWE				RLWE		BRK		TRK		pk&rlk		λ
	n	α	B_{ks}	d_{ks}	N	β	B_{br}	d_{br}	B	d	B_r	d_r	
I	560	3.05	2^2	8	2048	1.2	2^{12}	3	2^8	2	2^{10}	2	100
II	560	3.05	2^2	8	2048	1.2	2^{12}	3	2^8	2	2^{10}	2	100
III	560	3.05	2^2	8	2048	1.2	2^{12}	3	2^6	4	2^8	3	100
IV	560	3.05	2^2	8	2048	1.2	2^9	4	2^6	4	2^4	9	100
V	560	3.05	2^2	8	2048	1.2	2^9	4	2^5	8	2^4	10	100
I'	636	6.05	2^2	8	2048	4.4	2^{12}	3	2^8	2	2^{10}	2	128
II'	636	6.05	2^2	8	2048	4.4	2^{12}	3	2^8	2	2^{10}	2	128
III'	636	6.05	2^2	8	2048	4.4	2^{12}	3	2^6	4	2^8	3	128
IV'	636	6.05	2^2	8	2048	4.4	2^9	4	2^6	4	2^4	9	128
V'	636	6.05	2^2	8	2048	4.4	2^9	4	2^5	8	2^4	10	128

Table 4. Bootstrapping runtime (ms) on single-threaded CPU.

Scheme	Assump.	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
Security Level: $\lambda = 100$						
KMS [33]	RLWE	82	255	624	1541	4512
XZW+ [59]	NTRU	75	252	899	3377	–
HERDS	RLWE	43	86	181	471	1195
Security Level: $\lambda = 128$						
XZW+ [59]	NTRU	96	321	1152	4332	–
HERDS	RLWE	49	98	203	528	1351

5.2 Performance Results and Comparisons

Overall Latency. Table 4 reports a comparison of our scheme with the RLWE-based KMS [33] and the NTRU-based XZW+ [59]. At the 100-bit security level, our scheme achieves CPU runtimes of 43, 86, 181, 471, and 1195 ms for $k = 2, 4, 8, 16, 32$, achieving speedups of $1.7\times$ – $7.2\times$ over XZW+ and $1.9\times$ – $3.8\times$ over KMS (XZW+ does not provide results for $k = 32$). At the 128-bit security level, our scheme further achieves $2.0\times$ – $8.2\times$ improvements over XZW+ for $k = 2, 4, 8, 16$, while KMS does not report parameters at this level.

GPU Performance Breakdown. To further understand how well the parallelism inherent in our proposed scheme adapts to GPU architectures, we conduct a fine-grained performance profiling of the MK-FHE bootstrapping process. This profiling characterizes the time distribution of each component within the overall execution, as summarized in Fig. 8. The results indicate that the primary bottlenecks lie in the MK-LWE to RLWE and RLWE to MK-RLWE transformations, whereas the cost of homomorphic decoding is negligible. To enable efficient ci-

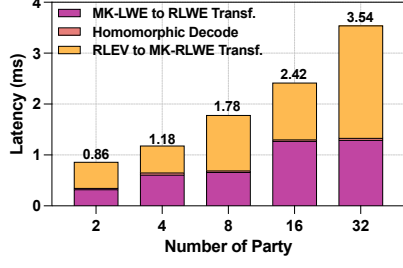


Fig. 6. *
(a) 100-bit security

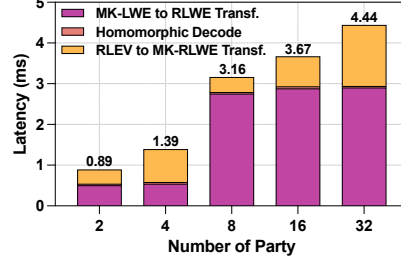


Fig. 7. *
(b) 128-bit security

Fig. 8. Bootstrapping runtime (ms) breakdown of the GPU implementation.

phertext conversion, we exploit the inherent inter-party parallelism by mapping it onto a GPU multi-stream architecture, where each party is assigned to an independent CUDA stream. This design decouples tasks and supports asynchronous scheduling, thereby improving computation overlap and maximizing GPU resource utilization. Details of the intra-party, parallel-friendly blind rotation are provided in Appendix C. By leveraging the algorithmic parallelism alongside the collaborative acceleration offered by the GPU architecture, HERDS achieves speedups of **50–337.6 $\times$** and **55–343 $\times$** over its CPU counterpart at the 100-bit and 128-bit security, respectively. For more detailed data on each critical path, please refer to Tab. 5. Concretely, we eliminate the gadget-length factor in the blind-rotation complexity, and further reduce the dependence on the ciphertext dimension from linear to sublinear via parallelization. Our single-threaded CPU results isolate the gains from these algorithmic optimizations, while the GPU results corroborate the sublinear scaling enabled by our parallel strategy.

Sensitivity Analysis on ω_2 . In the parallel-friendly blind rotation algorithm, the parameter ω_2 has a direct impact on performance. Intuitively, a larger ω_2 provides greater parallelism for the MK-LWE to RLWE blind rotation, thereby improving throughput. However, it also increases the computational cost of the subsequent RLWE to MK-RLWE transformation. Thus, selecting an appropriate value for ω_2 requires careful tuning to strike a balance between these opposing factors. To quantitatively evaluate this trade-off, we conduct a sensitivity analysis under both 100-bit and 128-bit security settings. As shown in Fig. 11, as ω_2 increases, the overall latency initially decreases, reaching its optimal point at $\omega_2 = \{16, 8, 8, 4, 4\}$ for 100-bit security and $\omega_2 = \{12, 12, 4, 4, 4\}$ for 128-bit security. Beyond these points, performance begins to degrade due to the growing overhead in the RLWE to MK-RLWE transformation stage.

Table 5. Bootstrapping runtime (ms) breakdown of the GPU implementation.

Operation	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
Security Level: $\lambda = 100$					
MK-LWE to RLWE Transf.	0.32	0.61	0.66	1.27	1.29
Homomorphic Decode	0.03	0.04	0.03	0.03	0.04
RLEV to MK-RLWE Transf.	0.51	0.53	1.09	1.12	2.21
Total Time	0.86	1.18	1.78	2.42	3.54
Security Level: $\lambda = 128$					
MK-LWE to RLWE Transf.	0.5	0.54	2.75	2.88	2.9
Homomorphic Decode	0.04	0.04	0.04	0.05	0.04
RLEV to MK-RLWE Transf.	0.35	0.81	0.37	0.74	1.5
Total Time	0.89	1.39	3.16	3.67	4.44

Table 6. Bootstrapping key size (MB).

Scheme	Assump.	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
Security Level: $\lambda = 100$						
HERDS	RLWE	105	105	105	140	141
KMS [33]	RLWE	105	175	140	175	211
XZW+ [59]	NTRU	13	13	13	13	–
Security Level: $\lambda = 128$						
HERDS	RLWE	119	119	119	159	160
XZW+ [59]	NTRU	17	17	17	25	–

Memory Consumption. Table 6 lists the bootstrapping key sizes of different schemes. NTRU-based schemes [59] have the smallest key sizes due to NTRU’s inherent efficiency and compact parameterization, but they cannot scale beyond 16 parties. Our scheme, while having larger keys than NTRU-based schemes, achieves smaller key sizes than existing RLWE-based approaches [33], as Block LWE provides better noise control, enabling the use of more compact parameters. Overall, the memory required for storing the bootstrapping keys is well within the capacity of the CPU and GPU environments we used in our experiments.

Acknowledgements We thank the anonymous reviewers of ASIACRYPT 2025 and EUROCRYPT 2026 for their helpful comments and suggestions on earlier versions of this paper. This work was supported by the National Key Research and Development Program of China (Grant No. 2023YFB4503203), the Strategic Priority Research Program of the Chinese Academy of Sciences (Grant No. XDB0690200), the National Natural Science Foundation of China (Grant Nos. 62525205, 62372447, 62125204, and 92270201), the Innovation Program for

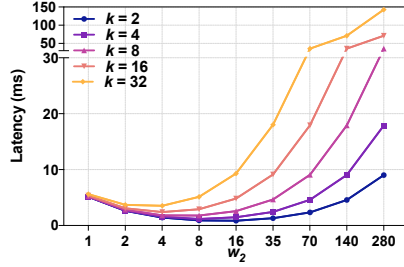


Fig. 9. *
(a) 100-bit security

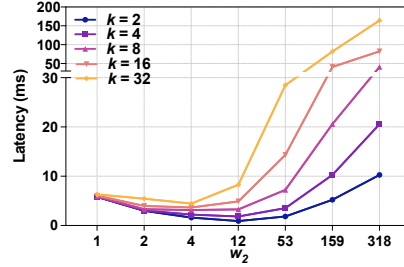


Fig. 10. *
(b) 128-bit security

Fig. 11. GPU performance under different numbers of participants k and different ω_2 values. Optimal GPU performance is attained when the number of parties $k = 2, 4, 8, 16, 32$ corresponds to (a) $\omega_2 = 16, 8, 8, 4, 4$ for 100-bit security, and (b) $\omega_2 = 12, 12, 4, 4, 4$ for 128-bit security.

Quantum Science and Technology (Grant Nos. 2021ZD0302901 and 2021ZD0302902), and the Open Fund of the State Key Laboratory of Cryptology (Grant No. MMK-FKT202508).

References

1. Akin, Y., Klemsa, J., Önen, M.: A practical tffe-based multi-key homomorphic encryption with linear complexity and low noise growth. In: ESORICS 2023. Lecture Notes in Computer Science, vol. 14344, pp. 3–23. Springer (2023). https://doi.org/10.1007/978-3-031-50594-2_1, https://doi.org/10.1007/978-3-031-50594-2_1
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. J. Math. Cryptol. **9**(3), 169–203 (2015), <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml>
3. Angel, S., Chen, H., Laine, K., Setty, S.T.V.: PIR with compressed queries and amortized query processing. In: 2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21–23 May 2018, San Francisco, California, USA. pp. 962–979. IEEE Computer Society (2018). <https://doi.org/10.1109/SP.2018.00062>, <https://doi.org/10.1109/SP.2018.00062>
4. Asharov, G., Jain, A., López-Alt, A., Tromer, E., Vaikuntanathan, V., Wichs, D.: Multiparty computation with low communication, computation and interaction via threshold FHE. In: EUROCRYPT 2012. Lecture Notes in Computer Science, vol. 7237, pp. 483–501. Springer (2012). https://doi.org/10.1007/978-3-642-29011-4_29, https://doi.org/10.1007/978-3-642-29011-4_29
5. Badawi, A.A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Quah, I., Polyakov, Y., Saraswathy, R.V., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: OpenFHE: Open-source fully homomorphic encryption

- library. In: Brenner, M., Costache, A., Rohloff, K. (eds.) *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, Los Angeles, CA, USA, 7 November 2022. pp. 53–63. ACM (2022). <https://doi.org/10.1145/3560827.3563379>, <https://doi.org/10.1145/3560827.3563379>
6. Belorgey, M.G., Carпов, S., Gama, N., Guasch, S., Jetchev, D.: Revisiting key decomposition techniques for FHE: simpler, faster and more generic. In: Chung, K., Sasaki, Y. (eds.) *ASIACRYPT 2024. Lecture Notes in Computer Science*, vol. 15484, pp. 176–207. Springer (2024). https://doi.org/10.1007/978-981-96-0875-1_6, https://doi.org/10.1007/978-981-96-0875-1_6
 7. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: *ACM Trans. Comput. Theory*. vol. 6, pp. 13:1–13:36 (2014). <https://doi.org/10.1145/2633600>, <https://doi.org/10.1145/2633600>
 8. Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) LWE. In: Ostrovsky, R. (ed.) *FOCS 2011*. pp. 97–106. IEEE Computer Society (2011). <https://doi.org/10.1109/FOCS.2011.12>, <https://doi.org/10.1109/FOCS.2011.12>
 9. Brenna, L., Singh, I.S., Johansen, H.D., Johansen, D.: TFHE-rs: A library for safe and secure remote computing using fully homomorphic encryption and trusted execution environments. *Array* **13**, 100118 (2022)
 10. Chen, H., Chillotti, I., Song, Y.: Multi-key homomorphic encryption from TFHE. In: *ASIACRYPT 2019. Lecture Notes in Computer Science*, vol. 11922, pp. 446–472. Springer (2019). https://doi.org/10.1007/978-3-030-34621-8_16
 11. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference. In: *CCS 2019*. pp. 395–412. ACM (2019). <https://doi.org/10.1145/3319535.3363207>, <https://doi.org/10.1145/3319535.3363207>
 12. Chen, H., Huang, Z., Laine, K., Rindal, P.: Labeled PSI from fully homomorphic encryption with malicious security. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) *CCS 2018*. pp. 1223–1237. ACM (2018). <https://doi.org/10.1145/3243734.3243836>, <https://doi.org/10.1145/3243734.3243836>
 13. Chen, H., Laine, K., Rindal, P.: Fast private set intersection from homomorphic encryption. In: Thuraisingham, B., Evans, D., Malkin, T., Xu, D. (eds.) *CCS 2017*. pp. 1243–1255. ACM (2017). <https://doi.org/10.1145/3133956.3134061>, <https://doi.org/10.1145/3133956.3134061>
 14. Chen, L., Zhang, Z., Wang, X.: Batched multi-hop multi-key FHE from ring-LWE with compact ciphertext extension. In: *TCC 2017. LNCS*, vol. 10678, pp. 597–627. Springer (2017). https://doi.org/10.1007/978-3-319-70503-3_20, https://doi.org/10.1007/978-3-319-70503-3_20
 15. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) *ASIACRYPT 2017. Lecture Notes in Computer Science*, vol. 10624, pp. 409–437. Springer (2017). https://doi.org/10.1007/978-3-319-70694-8_15, https://doi.org/10.1007/978-3-319-70694-8_15
 16. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In: *ASIACRYPT 2017. Lecture Notes in Computer Science*, vol. 10624, pp. 377–408. Springer (2017). https://doi.org/10.1007/978-3-319-70694-8_14, https://doi.org/10.1007/978-3-319-70694-8_14
 17. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: fast fully homomorphic encryption over the torus. *J. Cryptol.* **33**(1), 34–91 (2020). <https://doi.org/10.1007/S00145-019-09319-X>, <https://doi.org/10.1007/s00145-019-09319-x>

18. Clear, M., McGoldrick, C.: Multi-identity and multi-key leveled FHE from learning with errors. In: CRYPTO 2015. LNCS, vol. 9216, pp. 630–656. Springer (2015). https://doi.org/10.1007/978-3-662-48000-7_31, https://doi.org/10.1007/978-3-662-48000-7_31
19. Cong, K., Moreno, R.C., da Gama, M.B., Dai, W., Iliashenko, I., Laine, K., Rosenberg, M.: Labeled PSI from homomorphic encryption with reduced computation and communication. In: Kim, Y., Kim, J., Vigna, G., Shi, E. (eds.) CCS 21. pp. 1135–1150. ACM (2021). <https://doi.org/10.1145/3460120.3484760>, <https://doi.org/10.1145/3460120.3484760>
20. Dahl, M., Demmler, D., Kazdadi, S.E., Meyre, A., Orfila, J., Rotaru, D., Smart, N.P., Tap, S., Walter, M.: Noah’s ark: Efficient threshold-fhe using noise flooding. In: Brenner, M., Costache, A., Rohloff, K. (eds.) Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography, Copenhagen, Denmark, 26 November 2023. pp. 35–46. ACM (2023). <https://doi.org/10.1145/3605759.3625259>, <https://doi.org/10.1145/3605759.3625259>
21. Dalvi, A., Jain, A., Moradiya, S., Nirmal, R., Sanghavi, J., Siddavatam, I.: Securing neural networks using homomorphic encryption. In: 2021 International Conference on Intelligent Technologies (CONIT). pp. 1–7 (2021). <https://doi.org/10.1109/CONIT51480.2021.9498376>
22. Ducas, L., van Woerden, W.: NTRU fatigue: How stretched is over-stretched? In: ASIACRYPT 2021. LNCS, vol. 13093, pp. 3–32. Springer (2021). https://doi.org/10.1007/978-3-030-92068-5_1, https://doi.org/10.1007/978-3-030-92068-5_1
23. Fisch, B., Lazaretti, A., Liu, Z., Papamanthou, C.: Thorpir: Single server PIR via homomorphic thorp shuffles. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) CCS 2024. pp. 1448–1462. ACM (2024). <https://doi.org/10.1145/3658644.3690326>, <https://doi.org/10.1145/3658644.3690326>
24. Friedman, O., Marmor, A., Mutzari, D., Scalý, Y.C., Spiizer, Y., Yanai, A.: Tiresias: Large scale, uc-secure threshold paillier. In: Chung, K., Sasaki, Y. (eds.) ASIACRYPT 2024. vol. 15486, pp. 141–173. Springer (2024). https://doi.org/10.1007/978-981-96-0891-1_5, https://doi.org/10.1007/978-981-96-0891-1_5
25. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: STOC 2009. pp. 169–178 (2009). <https://doi.org/10.1145/1536414.1536440>
26. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013. vol. 8042, pp. 75–92. Springer (2013). https://doi.org/10.1007/978-3-642-40041-4_5, https://doi.org/10.1007/978-3-642-40041-4_5
27. Goldwasser, S., Kalai, Y.T., Peikert, C., Vaikuntanathan, V.: Robustness of the learning with errors assumption. In: Innovations in Computer Science - ICS 2010, Tsinghua University, Beijing, China, January 5-7, 2010. Proceedings. pp. 230–240. Tsinghua University Press (2010), <http://conference.iis.tsinghua.edu.cn/ICS2010/content/papers/19.html>
28. Hwang, I.: TFHE-go. Online: <https://github.com/sp301415/tfhe-go> (2023)
29. Juvekar, C., Vaikuntanathan, V., Chandrakasan, A.P.: GAZELLE: A low latency framework for secure neural network inference. In: Enck, W., Felt, A.P. (eds.) USENIX Security 2018. pp. 1651–1669. USENIX Association (2018), <https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar>
30. Kim, J., Lee, C.: A polynomial time algorithm for breaking NTRU encryption with multiple keys. Des. Codes Cryptogr. **91**(8), 2779–2789 (2023). <https://doi.org/10.1007/S10623-023-01233-5>, <https://doi.org/10.1007/s10623-023-01233-5>

31. Kim, T., Kwak, H., Lee, D., Seo, J., Song, Y.: Asymptotically faster multi-key homomorphic encryption from homomorphic gadget decomposition. In: CCS 2023. pp. 726–740. ACM (2023). <https://doi.org/10.1145/3576915.3623176>, <https://doi.org/10.1145/3576915.3623176>
32. Kraitsberg, M., Lindell, Y., Osheter, V., Smart, N.P., Alaoui, Y.T.: Adding distributed decryption and key generation to a ring-lwe based CCA encryption scheme. In: Jang-Jaccard, J., Guo, F. (eds.) ACISP 2019. vol. 11547, pp. 192–210. Springer (2019). https://doi.org/10.1007/978-3-030-21548-4_11, https://doi.org/10.1007/978-3-030-21548-4_11
33. Kwak, H., Min, S., Song, Y.: Towards practical multi-key TFHE: parallelizable, key-compatible, quasi-linear complexity. In: PKC 2024. Lecture Notes in Computer Science, vol. 14604, pp. 354–385. Springer (2024). https://doi.org/10.1007/978-3-031-57728-4_12
34. Lee, C., Min, S., Seo, J., Song, Y.: Faster TFHE bootstrapping with block binary keys. In: Liu, J.K., Xiang, Y., Nepal, S., Tsudik, G. (eds.) ASIA CCS 2023. pp. 2–13. ACM (2023). <https://doi.org/10.1145/3579856.3595804>, <https://doi.org/10.1145/3579856.3595804>
35. Lee, J.W., Kang, H., Lee, Y., Choi, W., Eom, J., Deryabin, M., Lee, E., Lee, J., Yoo, D., Kim, Y.S., No, J.S.: Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. IEEE Access **10**, 30039–30054 (2022). <https://doi.org/10.1109/ACCESS.2022.3159694>
36. Lee, J., Lee, E., Kim, Y., No, J.: Rotation key reduction for client-server systems of deep neural network on fully homomorphic encryption. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023. vol. 14443, pp. 36–68. Springer (2023). https://doi.org/10.1007/978-981-99-8736-8_2, https://doi.org/10.1007/978-981-99-8736-8_2
37. Lee, K., Yeo, Y.: Sophomr: Improved oblivious message retrieval from simd-aware homomorphic compression. IACR Cryptol. ePrint Arch. p. 1814 (2024), <https://eprint.iacr.org/2024/1814>
38. Lee, Y., Micciancio, D., Kim, A., Choi, R., Deryabin, M., Eom, J., Yoo, D.: Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023. Lecture Notes in Computer Science, vol. 14006, pp. 227–256. Springer (2023). https://doi.org/10.1007/978-3-031-30620-4_8, https://doi.org/10.1007/978-3-031-30620-4_8
39. Liu, Z., Tromer, E.: Oblivious message retrieval. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022. Lecture Notes in Computer Science, vol. 13507, pp. 753–783. Springer (2022). https://doi.org/10.1007/978-3-031-15802-5_26, https://doi.org/10.1007/978-3-031-15802-5_26
40. Liu, Z., Tromer, E., Wang, Y.: Group oblivious message retrieval. In: IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19–23, 2024. pp. 4367–4385. IEEE (2024). <https://doi.org/10.1109/SP54263.2024.00115>, <https://doi.org/10.1109/SP54263.2024.00115>
41. López-Alt, A., Tromer, E., Vaikuntanathan, V.: On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: STOC 2012. pp. 1219–1234. ACM (2012), <https://doi.org/10.1145/2213977.2214086>
42. Lu, W., Huang, Z., Hong, C., Ma, Y., Qu, H.: PEGASUS: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In: SP 2021. pp. 1057–1073. IEEE (2021). <https://doi.org/10.1109/SP40001.2021.00043>, <https://doi.org/10.1109/SP40001.2021.00043>

43. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: EUROCRYPT 2010. pp. 1–23. Springer (2010). https://doi.org/10.1007/978-3-642-13190-5_1
44. Menon, S.J., Wu, D.J.: SPIRAL: fast, high-rate single-server PIR via FHE composition. In: 43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22–26, 2022. pp. 930–947. IEEE (2022). <https://doi.org/10.1109/SP46214.2022.9833700>, <https://doi.org/10.1109/SP46214.2022.9833700>
45. Micciancio, D., Suhl, A.: Simulation-secure threshold PKE from LWE with polynomial modulus. IACR Commun. Cryptol. **1**(4), 2 (2024). <https://doi.org/10.62056/AOZOGY4E->, <https://doi.org/10.62056/a0zogy4e->
46. Micciancio, D., Suhl, A.: Simulation-secure threshold PKE from LWE with polynomial modulus. IACR Commun. Cryptol. **1**(4), 2 (2024). <https://doi.org/10.62056/AOZOGY4E->, <https://doi.org/10.62056/a0zogy4e->
47. Mouchet, C., Bertrand, E., Hubaux, J.: An efficient threshold access-structure for RLWE-based multiparty homomorphic encryption. J. Cryptol. **36**(2), 10 (2023). <https://doi.org/10.1007/S00145-023-09452-8>, <https://doi.org/10.1007/s00145-023-09452-8>
48. Mouchet, C., Troncoso-Pastoriza, J.R., Bossuat, J., Hubaux, J.: Multiparty homomorphic encryption from ring-learning-with-errors. Proc. Priv. Enhancing Technol. **2021**(4), 291–311 (2021). <https://doi.org/10.2478/POPETS-2021-0071>, <https://doi.org/10.2478/popets-2021-0071>
49. Mukherjee, P., Wichs, D.: Two round multiparty computation via multi-key FHE. In: EUROCRYPT 2016. LNCS, vol. 9666, pp. 735–763. Springer (2016). https://doi.org/10.1007/978-3-662-49896-5_26, https://doi.org/10.1007/978-3-662-49896-5_26
50. Okada, H., Takagi, T.: Low communication threshold FHE from standard (module-)lwe. In: Hanaoka, G., Yang, B. (eds.) ASIACRYPT 2025. Lecture Notes in Computer Science, vol. 16251, pp. 165–198. Springer (2025). https://doi.org/10.1007/978-981-95-5122-4_6, https://doi.org/10.1007/978-981-95-5122-4_6
51. Park, J.: Homomorphic encryption for multiple users with less communications. IEEE Access **9**, 135915–135926 (2021). <https://doi.org/10.1109/ACCESS.2021.3117029>, <https://doi.org/10.1109/ACCESS.2021.3117029>
52. Park, J., Leeuwen, B.V., Zajonc, O.: FINALLY: A multi-key FHE scheme based on NTRU and LWE. IACR Commun. Cryptol. **1**(3), 15 (2024). <https://doi.org/10.62056/AEBN-4C2H>, <https://doi.org/10.62056/aebn-4c2h>
53. Park, J., Rovira, S.: Efficient TFHE bootstrapping in the multiparty setting. IEEE Access **11**, 118625–118638 (2023). <https://doi.org/10.1109/ACCESS.2023.3326237>, <https://doi.org/10.1109/ACCESS.2023.3326237>
54. Passelègue, A., Stehlé, D.: Low communication threshold fully homomorphic encryption. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024. Lecture Notes in Computer Science, vol. 15484, pp. 297–329. Springer (2024). https://doi.org/10.1007/978-981-96-0875-1_10, https://doi.org/10.1007/978-981-96-0875-1_10
55. Peikert, C., Shiehian, S.: Multi-key FHE from LWE, revisited. In: TCC 2016-B. LNCS, vol. 9986, pp. 217–238 (2016). https://doi.org/10.1007/978-3-662-53644-5_9, https://doi.org/10.1007/978-3-662-53644-5_9
56. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM (JACM) **56**(6), 1–40 (2009). <https://doi.org/10.1145/1568318.1568324>

57. Sugizaki, Y., Tsuchida, H., Hayashi, T., Nuida, K., Nakashima, A., Isshiki, T., Mori, K.: Threshold fully homomorphic encryption over the torus. In: Tsudik, G., Conti, M., Liang, K., Smaragdakis, G. (eds.) Computer Security - ESORICS 2023. Lecture Notes in Computer Science, vol. 14344, pp. 45–65. Springer (2023). https://doi.org/10.1007/978-3-031-50594-2_3, https://doi.org/10.1007/978-3-031-50594-2_3
58. Wang, R., Ha, J., Shen, X., Lu, X., Chen, C., Wang, K., Lee, J.: FHEW-like leveled homomorphic evaluation: Refined workflow and polished building blocks. Cryptology ePrint Archive, Paper 2024/1318 (2024), <https://eprint.iacr.org/2024/1318>
59. Xiang, B., Zhang, J., Wang, K., Deng, Y., Feng, D.: NTRU-based bootstrapping for MK-FHEs without using overstretched parameters. In: ASIACRYPT 2024. Lecture Notes in Computer Science, vol. 15484, pp. 241–270. Springer (2024). https://doi.org/10.1007/978-981-96-0875-1_8, https://doi.org/10.1007/978-981-96-0875-1_8
60. Xu, K., Tan, B.H.M., Wang, L., Aung, K.M.M., Wang, H.: Multi-key fully homomorphic encryption from NTRU and (R)LWE with faster bootstrapping. Theor. Comput. Sci. **968**, 114026 (2023). <https://doi.org/10.1016/J.TCS.2023.114026>, <https://doi.org/10.1016/j.tcs.2023.114026>

Supplementary Material

A TFHE Scheme

In this section, we provide a brief overview of the TFHE scheme [17].

Single-key TFHE

• Setup(1^λ) : For the security parameter λ , choose the following parameters:

LWE dimension n , RLWE dimension N , noise parameters $\alpha, \beta > 0$, the gadget decomposition parameters B, d , and B', d' , corresponding to the gadget vectors $\mathbf{g}, \mathbf{g}'$ and decomposition functions h, h' respectively, for the RGSW cryptosystem, and LWE key-switching, respectively. Output the public parameters $\mathbf{pp} = (n, N, \alpha, \beta, B, d, B', d')$.

• KeyGen($\mathbf{pp}$) : Given the public parameters $\mathbf{pp} = (n, N, \alpha, \beta, B, d, B', d')$, do the following:

- Sample $z_i \leftarrow \mathcal{U}(\{0, 1\})$ for $0 \leq i < n$, and output $\mathbf{z} = (z_0, \dots, z_{n-1}) \in \mathbb{Z}^n$.
- Sample $s_i \leftarrow \mathcal{U}(\{0, 1\})$ for $0 \leq i < N$, and output $s = \sum_{i=0}^{N-1} s_i X^i \in R$.
- Set $\mathbf{brk}_i \leftarrow \text{RGSW.Enc}(z_i, s)$ under the gadget parameters B and d and the noise distribution $\mathcal{D}_\beta$, for $0 \leq i < n$. Return $\mathbf{BRK} = \{\mathbf{brk}_i\}_{0 \leq i < n}$.
- Sample $\mathbf{A}_i \leftarrow \mathcal{U}(\mathbb{Z}_q^{d' \times n})$ and $\mathbf{e}_i \leftarrow \mathcal{D}_\alpha^{d'}$ for $0 \leq i < N$, and set $\mathbf{ksk}_i = [\mathbf{b}_i \mid \mathbf{A}_i] \in \mathbb{Z}_q^{d' \times (n+1)}$ where $\mathbf{b}_i = -\mathbf{A}_i \cdot \mathbf{z} + z_i \cdot \mathbf{g}' + \mathbf{e}_i \pmod{q}$. Return $\mathbf{KSK} = \{\mathbf{ksk}_i\}_{0 \leq i < N}$.

In the following, we write $\mathbf{s}$ to denote the vectorized ring key, i.e., $\mathbf{s} = (s_0, \dots, s_{N-1}) \in \mathbb{Z}^N$.

• Enc($m, \mathbf{z}$) : Samples $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ and $e \leftarrow \mathcal{D}_\alpha$. Given a message $m \in \{0, 1\}$ and a secret $\mathbf{z}$, compute and return the ciphertext $(b, \mathbf{a}) \in \mathbb{Z}_q^{n+1}$ where $b = -\langle \mathbf{a}, \mathbf{z} \rangle + \lfloor \frac{q}{4} m \rfloor - \frac{1}{8} + e \pmod{q}$.

• Dec($(b, \mathbf{a}), \mathbf{z}$) : Given a ciphertext $(b, \mathbf{a}) \in \mathbb{Z}_q^{n+1}$, compute $\mu = b + \langle \mathbf{a}, \mathbf{z} \rangle \pmod{q}$. Output 0 if $\mu < 0$, and output 1 otherwise.

• HomNAND($(b_1, \mathbf{a}_1), (b_2, \mathbf{a}_2), \mathbf{BRK}, \mathbf{KSK}$) : Given two ciphertexts $(b_1, \mathbf{a}_1) \in \mathbb{Z}_q^{n+1}$ and $(b_2, \mathbf{a}_2) \in \mathbb{Z}_q^{n+1}$, and the blind rotation and key-switching keys $\mathbf{BRK}$ and $\mathbf{KSK}$, perform the following:

- Compute $(b, \mathbf{a}) = (\lfloor \frac{q}{8} \rfloor, \mathbf{0}) - (b_1, \mathbf{a}_1) - (b_2, \mathbf{a}_2) \pmod{q}$.
- Run Algorithm 5 with inputs $(b, \mathbf{a})$ and $\mathbf{BRK}$, and let $\mathbf{ACC} \in R_q^2$ be the output.
- Parse $\mathbf{ACC} = (\sum_{i=0}^{N-1} b'_i X^i, \sum_{i=0}^{N-1} a'_i X^i)$, and perform the key-switching operation as follows, and return the output LWE ciphertext:

$$(b', \mathbf{0}) + h'(a_0)^\top \cdot \mathbf{ksk}_0 + \sum_{i=1}^{N-1} h'(-a_{N-i})^\top \cdot \mathbf{ksk}_i \pmod{q}.$$

Algorithm 5 RLWE blind rotation from LWE

Input:

The LWE ciphertext $(b, \mathbf{a}) \in \mathbb{Z}_q^{n+1}$ where $\mathbf{a} = (a_0, \dots, a_{n-1}) \in \mathbb{Z}_q^n$

The blind rotation key BRK

Output:

An RLWE ciphertext $(b, a) \in R_q^2$.

```

1:  $\hat{b} \leftarrow \left\lfloor \left\lfloor \frac{b \cdot 2N}{q} \right\rfloor \right\rfloor_{2N}$ 
2: for  $i = 0$  to  $n - 1$  do
3:    $\hat{a}_i \leftarrow \left\lfloor \left\lfloor \frac{a_i \cdot 2N}{q} \right\rfloor \right\rfloor_{2N}$ 
4:  $\text{ACC} \leftarrow (-\lfloor \frac{q}{8} \rfloor X^{-N/2+\hat{b}} \cdot \sum_{i=0}^{N-1} X^i, 0) \in R_q^2$ 
5: for  $i = 0$  to  $n - 1$  do
6:    $\text{ACC} \leftarrow \text{ACC} + (X^{\hat{a}_i} - 1) \cdot \text{ACC} \boxtimes \text{brk}_i \pmod{q}$ 
7: return ACC
  
```

B Proofs

B.1 Proof of Lemma 1

Proof. Let $\mathbf{g} \in \mathbb{Z}^d$ and $\mathbf{g}^{-1} : R_q \rightarrow R^d$ be the corresponding gadget vector and the decomposition function to the parameters $B, d > 0$. Also, let us denote the noise vector of $C = (c_{i,j})_{0 \leq i < d, 0 \leq j \leq 1}$ by $(e_0, \dots, e_{d-1})^\top$. Then, for $\mathbf{g}^{-1}(a) = (a_0, \dots, a_{d-1})$, the scalar multiplication yields:

$$\begin{aligned}
 \varphi_s(a \odot C) &= \varphi_s(\mathbf{g}^{-1}(a)^\top \cdot C) = \sum_{i=0}^{d-1} a_i \cdot \varphi_s((c_{i,0}, c_{i,1})) \\
 &= \sum_{i=0}^{d-1} a_i \cdot (g_i \mu + e_i) = a \cdot \mu + \sum_{i=0}^{d-1} a_i e_i + (a - \langle \mathbf{g}, \mathbf{g}^{-1}(a) \rangle) \cdot \mu.
 \end{aligned}$$

Recall that heuristically each coefficient of a_i follows a uniform random distribution over $(-B/2, B/2) \cap \mathbb{Z}$, the variance of $\sum_{i=0}^{d-1} a_i e_i$ is given by $dN \frac{B^2}{12} \sigma_{in}^2$. Also, recall that each coefficient of $a - \langle \mathbf{g}, \mathbf{g}^{-1}(a) \rangle$ follows a uniform distribution over $(-\epsilon, \epsilon) \cap \mathbb{Z}$, the variance of the final term is bounded by $\|\mu\|_1 \cdot \frac{\epsilon^2}{3}$. The total noise variance bound follows by combining these terms. $\square$

B.2 Proof of Lemma 2

Proof. By definition, $b + as = m + e_{RLWE} \pmod{q}$ for some small e_{RLWE} whose variance is σ_{RLWE}^2 . On the other hand, we can parse the input RGSW ciphertext $\overline{C} = [C_0 \ C_1]^\top \in R_q^{2d \times 2}$ for two RLEV encryptions $C_0, C_1 \in R_q^{d \times 2}$ of μ and $\mu \cdot s$, as noted above. Then, by definition, $c \boxtimes \overline{C} = b \odot C_0 + a \odot C_1$. By Lemma 2, we have

$$\varphi_s(b \odot C_0) + \varphi_s(a \odot C_1) = b \cdot \mu + e_b + a \cdot \mu s + e_a$$

$$= \mu \cdot m + \mu \cdot e_{RLWE} + e_b + e_a \pmod{q}$$

for some small e_a and e_b , each having noise bounds of $dN \frac{B^2}{12} \sigma_{RGSW}^2 + \|\mu\|_1 \cdot \frac{\epsilon^2}{3}$ and $dN \frac{B^2}{12} \sigma_{RGSW}^2 + \|\mu \cdot s\|_1 \cdot \frac{\epsilon^2}{3}$, respectively. Therefore, the final noise $\mu \cdot e_{RLWE} + e_b + e_a$ has a variance $\sigma_{\square}^2$ bounded by

$$\|\mu\|_1 \cdot \sigma_{RLWE}^2 + dN \frac{B^2}{6} \sigma_{RGSW}^2 + (\|\mu\|_1 + \|\mu s\|_1) \frac{\epsilon^2}{3}.$$

□

B.3 Proof of Lemma 3

Proof. Suppose that $\varphi(c) = m + e \pmod{q}$ for the plaintext $m \in R_q$ and noise $e \in R$. Then,

$$\begin{aligned} \varphi(c') &= \lfloor \frac{q'}{q} b \rfloor + \lfloor \frac{q'}{q} a \rfloor \cdot s \\ &= \frac{q'}{q} (m + e) + e_{rnd} \pmod{q'} \end{aligned}$$

where

$$e_{rnd} = \left(\lfloor \frac{q'}{q} \cdot b \rfloor - \frac{q'}{q} \cdot b \right) + \left(\lfloor \frac{q'}{q} \cdot a \rfloor - \frac{q'}{q} \cdot a \right) \cdot s$$

Heuristically each coefficient of b and a is uniformly distributed in $\mathbb{Z}_q$, each coefficient of terms $\left(\lfloor \frac{q'}{q} \cdot b \rfloor - \frac{q'}{q} \cdot b \right)$ and $\left(\lfloor \frac{q'}{q} \cdot a \rfloor - \frac{q'}{q} \cdot a \right)$ follows a uniform distribution in $(-0.5, 0.5]$, hence having the variance of $1/12$. Therefore, the additional noise e_{rnd} introduced by the modulus switching has the variance σ_{ms}^2 bounded by $\frac{N+1}{12}$. Therefore, the final noise is bounded by $(\frac{q'}{q})^2 \cdot \sigma_{in}^2 + \sigma_{ms}^2$. □

B.4 Proof of Lemma 4

Proof. By definition, $b + as = m + e_{RLWE} \pmod{q}$ for some small e_{RLWE} whose variance is σ_{RLWE}^2 . Then by definition and Lemma 1, we have

$$\varphi_{s'}(c') = b + as + e_{ks} = m + e + e_{ks} \pmod{q}$$

for some small e_{ks} , whose variance σ_{ks}^2 is bounded by $\sigma_{ks}^2 \leq dN \cdot \frac{B^2}{12} \cdot \sigma_{RLEV}^2 + N \cdot \frac{\epsilon^2}{3}$ with $\|s\| \leq N$. □

B.5 Proof of Lemma 5

Proof. Algorithms 2 and 3 follow the same computational procedure, differing only in that the former employs a binary-tree evaluation and the latter a parallel evaluation. Hence, the output ciphertext noise is identical. For conciseness, we

present the full proof for Algorithm 3; the proof of Algorithm 2 is omitted and left to the reader.

Let $\text{ACC}_{i,j}$ be the RLWE ciphertext obtained during the j -th iteration of the inner loop within the i -th iteration of the outer accumulator loop. First, observe that line 8 of Algorithm 1 is equivalent to computing

$$\text{ACC}_{i,j+1} \leftarrow \text{ACC}_i + \text{ACC}_i \boxdot \left(\sum_{w=0}^{\ell-1} (X^{\hat{a}_{i,j\ell+w}} - 1) \text{brk}_{i,j\ell+w} \right)$$

due to the linearity of RGSW encryption, where $\hat{a}_i = (\hat{a}_{i,0}, \dots, \hat{a}_{i,n-1})$. Then, by definition 5 and algorithm 1, $\text{brk}'_{i,j} := \sum_{w=0}^{\ell-1} (X^{\hat{a}_{i,j\ell+w}} - 1) \text{brk}_{i,j\ell+w}$ is an RGSW encryption of $X^{a'_{i,j}} - 1$ for $a'_{i,j} := \sum_{w=0}^{\ell-1} \hat{a}_{i,j\ell+w} \cdot s_{i,j\ell+w}$, and its noise variance is $2\ell\sigma_{brk}^2$. Now, we have the following recurrence relation

$$\begin{aligned} \varphi_{s_i}(\text{ACC}_{i,j+1}) &= \varphi_{s_i}(\text{ACC}_{i,j}) + \varphi_{s_i}(\text{ACC}_{i,j} \boxdot \text{brk}'_{i,j}) \\ &= \varphi_{s_i}(\text{ACC}_{i,j}) + (X^{a'_{i,j}} - 1) \cdot \varphi_{s_i}(\text{ACC}_{i,j}) + e'_{i,j} \\ &= X^{a'_{i,j}} \cdot \varphi_{s_i}(\text{ACC}_{i,j}) + e'_{i,j} \pmod{q} \end{aligned}$$

for some small $e'_{i,j}$, whose variance is bounded by

$$\sigma_{e'_{i,j}}^2 \leq d_{br} N \frac{B_{br}^2}{6} \cdot 2\ell\sigma_{brk}^2 + (2 + 2N) \frac{\epsilon_{br}^2}{3}$$

by Lemma 2. Now, the above recurrence relation implies

$$\begin{aligned} \varphi_{s_i}(\text{ACC}_{i,n/\ell-1}) &= X^{a'_{i,n/\ell-1}} \cdot \varphi_{s_i}(\text{ACC}_{i,n/\ell-2}) + e'_{i,n/\ell-1} \\ &= X^{a'_{i,n/\ell-1} + a'_{i,n/\ell-2}} \cdot \varphi_{s_i}(\text{ACC}_{i,n/\ell-3}) + X^{a'_{i,n/\ell-2}} \cdot e'_{i,n/\ell-1} + e'_{i,n/\ell-2} \\ &= \dots = X^{\sum_{j=0}^{n/\ell-1} a'_{i,j}} \cdot \sum_{j=0}^{d-1} g_j X^j + \sum_{j=0}^{n/\ell-1} X^{\sum_{w=0}^{j-1} a'_{i,w}} \cdot e'_{i,j} \pmod{q} \end{aligned}$$

Note that the multiplication by a monomial only changes the position of the coefficients up to sign, we conclude that the final noise term $\sum_{j=0}^{n/\ell-1} X^{\sum_{w=0}^{j-1} a'_{i,w}} \cdot e'_{i,j}$ is bounded by

$$\frac{1}{3} d_{br} n N B_{br}^2 \sigma_{brk}^2 + \frac{2n}{3\ell} (N+1) \epsilon_{br}^2.$$

□

B.6 Proof of Lemma 6

Proof. Let us denote the output at the w -th iteration in Line 5 of Algorithm 3 by $c_{i,j,w}$, and write $\varphi_{s_i}(c_{i,j,0}) = \sum_{\kappa=0}^{N-1} v_{j,\kappa} X^\kappa$, where $c_{i,j,0} := X^{-j} \cdot c_{i,j}$. Now, assume that the phase of the ciphertext $c_{i,j,t}$ is given by

$$\varphi_{s_i}(c_{i,j,t}) = 2^t \cdot \sum_{\kappa=0}^{N/2^t-1} v_{j,2^t\kappa} X^{2^t\kappa} + \hat{e}'_t$$

for some small noise $\hat{e}'_t \in R$, with variance bounded by $\frac{4^t-1}{3} \cdot \sigma_{ks}^2$. Then, by definition and Lemma 4,

$$\begin{aligned}\varphi_{s_i}(c_{i,j,t+1}) &= \varphi_{s_i}\left(c_{i,j,t} + \text{KS}\left(\phi_{\frac{N}{2^{t+1}}+1}(c_{i,j,t}), \text{trk}_{i,t}\right)\right) \\ &= 2^{t+1} \cdot \sum_{\kappa=0}^{N/2^{t+1}-1} v_{j,2^{t+1}\kappa} X^{2^{t+1}\kappa} + \hat{e}'_t + \hat{e}'_t(X^{\frac{N}{2^{t+1}}+1}) + e_{ks} \pmod{q}\end{aligned}$$

for the key-switching noise $e_{ks} \in R$, with variance bounded by σ_{ks}^2 . As the automorphism operation permutes the coefficients up to sign, the ℓ_∞ -norm of $\hat{e}'_t + \hat{e}'_t(X^{\frac{N}{2^{t+1}}+1})$ is twice as large as that of $\hat{e}'_t$, the final noise $\hat{e}'_{t+1} := \hat{e}'_t + \hat{e}'_t(X^{\frac{N}{2^{t+1}}+1}) + e_{ks}$ has the noise variance bounded by $(4 \cdot \frac{4^t-1}{3} + 1) \cdot \sigma_{ks}^2 = \frac{4^{t+1}-1}{3} \cdot \sigma_{ks}^2$. Therefore, by mathematical induction, we conclude that

$$\varphi_{s_i}(c_{i,j,\log_2 d}) = d \cdot \sum_{\kappa=0}^{N/d-1} v_{j,d\kappa} X^{d\kappa} = g_j X^{\sum_{\kappa=0}^{n-1} \hat{a}_{i,\kappa} z_{i,\kappa}} + e_{i,j} \pmod{q}$$

for some small noise

$$e_{i,j} = d \sum_{\kappa=0}^{N/d-1} \left(\frac{1}{d} X^{-j} e_{i,d\kappa} X^{d\kappa} + X^{-j} e_{ms,i,d\kappa} X^{d\kappa} \right) + \hat{e}'_{\log_2 d}$$

where $e_{i,d\kappa}$ and $e_{ms,i,d\kappa}$ denote the $d\kappa$ -th coefficients of e_i and $e_{ms,i}$ respectively and the noise polynomial $\hat{e}'_{\log_2 d}$ has the variance bounded by $\frac{d^2-1}{3} \cdot \sigma_{ks}^2$. Note that the first equation follows from the fact $d \mid \sum_{\kappa=0}^{n-1} \hat{a}_{i,\kappa} z_{i,\kappa}$. As each ciphertext $c_{i,j}$ in Step 5 of the algorithm 3 is an RLWE encryption of $g_j X^{\sum_{\kappa=0}^{n-1} \hat{a}_{i,\kappa} z_{i,\kappa}}$, C_i is an RLEV ciphertext of $X^{\sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}}$. Also, by definition, Lemma 3, Lemma 4, and Theorem 5, the variance of e'' satisfies the bound:

$$\sigma_{e''}^2 \leq \sigma_{br}^2 + d^2 \sigma_{ms}^2 + \frac{d^2-1}{3} \cdot \sigma_{ks}^2,$$

where $\sigma_{br}^2 \leq \frac{1}{3} d_{br} n N B_{br}^2 \sigma_{brk}^2 + \frac{2n}{3\ell} (N+1) \epsilon_{br}^2$, $\sigma_{ms}^2 \leq \frac{N+1}{12}$, and $\sigma_{ks}^2 \leq d_{tr} N \cdot \frac{B_{tr}^2}{12} \cdot \sigma_{trk}^2 + N \cdot \frac{\epsilon_{tr}^2}{3}$ for $\epsilon_{br} = q/B_{br}^{d_{br}}$ and $\epsilon_{tr} = q/B_{tr}^{d_{tr}}$. $\square$

B.7 Proof of Lemma 7

Proof. To simplify the analysis, we neglect the noise introduced by the gadget decomposition with $\mathbf{g}_r$.¹¹ Let $\bar{c}_0 = (r(X)X^{\hat{b}}, 0, 0 \dots, 0)$, and $\bar{c}_i = (c_{i,0}, \dots, c_{i,k})$ denote the MK-RLWE ciphertext generated during the i -th iteration of the outer

¹¹ Indeed, we can set the gadget decomposition parameter extreme so that there is no approximate gadget parameter noise at all, because the latency of layer 2 is almost negligible in the whole process.

accumulator loop, and let $(w_{i,j}, u_{i,j})$ be the values obtained in the j -th inner iteration of line 4 from Algorithm 4 during the i -th outer iteration. In the following, we set $s_0 = 1$ for simplicity. Following the definitions and Lemma 1, we can obtain the following relation

$$\begin{aligned}\varphi_{\bar{s}}(\bar{c}_i) &= \sum_{j=0}^{i-1} (\langle \mathbf{g}_r^{-1}(u_j), \mathbf{d}_i \rangle + w_j) s_j + \langle \mathbf{g}_r^{-1}(v), \mathbf{a}'_2 \rangle + \langle \mathbf{g}_r^{-1}(v), \mathbf{f}_i \rangle \cdot s_i \\ &= \underbrace{\sum_{j=0}^{i-1} (\langle \mathbf{g}_r^{-1}(u_{i,j}), r_i \cdot \mathbf{a}'_1 + s_i \cdot \mathbf{g}_r + \mathbf{e}_{i,1} \rangle + w_{i,j}) s_j}_{\text{Term I}} + \underbrace{\langle \mathbf{g}_r^{-1}(v), \mathbf{a}'_2 s_i + \mathbf{f}_i \rangle}_{\text{Term II}} \pmod{q}.\end{aligned}$$

Then, the first term can be written as

$$\begin{aligned}\text{Term I} &= \sum_{j=0}^{i-1} (u_{i,j} s_i + w_{i,j}) s_j + \sum_{j=0}^{i-1} \langle \mathbf{g}_r^{-1}(u_{i,j}), (r_i \cdot \mathbf{a}'_1 + \mathbf{e}_{i,1}) \cdot s_j \rangle \\ &= \sum_{j=0}^{i-1} \varphi_{s_i}(c_{i-1,j} \odot C_i) s_j + \sum_{j=0}^{i-1} \langle \mathbf{g}_r^{-1}(u_{i,j}), (r_i \cdot \mathbf{a}'_1 + \mathbf{e}_{i,1}) \cdot s_j \rangle \\ &= X^{\sum_{j=0}^{i-1} (\hat{a}_{i,j} z_{i,j})} \varphi_{\bar{s}}(\bar{c}_{i-1}) + \sum_{j=0}^{i-1} e_j^{\odot} s_j + \sum_{j=0}^{i-1} \langle \mathbf{g}_r^{-1}(u_{i,j}), (r_i \cdot \mathbf{a}'_1 + \mathbf{e}_{i,1}) \cdot s_j \rangle \pmod{q}\end{aligned}$$

for the noise $e_j^{\odot}$ from the gadget products $c_{i-1,j} \odot C_i$. On the other hand, we have

$$\begin{aligned}\text{Term II} &= \langle \mathbf{g}_r^{-1}(v), \mathbf{g}_r \cdot r_i + \mathbf{e}_{i,2} \rangle = r_i v + \langle \mathbf{g}_r^{-1}(v), \mathbf{e}_{i,2} \rangle \\ &= r_i \sum_{j=0}^{i-1} \langle \mathbf{g}_r^{-1}(u_{i,j}), -\mathbf{a}'_1 s_j + \mathbf{e}_{j,0} \rangle + \langle \mathbf{g}_r^{-1}(v), \mathbf{e}_{i,2} \rangle \\ &= \sum_{j=0}^{i-1} \langle \mathbf{g}_r^{-1}(u_{i,j}), -\mathbf{a}'_1 s_j r_i \rangle + \sum_{j=0}^{i-1} \langle \mathbf{g}_r^{-1}(u_{i,j}), r_i \mathbf{e}_{j,0} \rangle + \langle \mathbf{g}_r^{-1}(v), \mathbf{e}_{i,2} \rangle \pmod{q}.\end{aligned}$$

Thus we have

$$\begin{aligned}\varphi_{\bar{s}}(\bar{c}_i) &= \text{Term I} + \text{Term II} \\ &= X^{\sum_{j=0}^{i-1} (\hat{a}_{i,j} z_{i,j})} \varphi_{\bar{s}}(\bar{c}_{i-1}) + e_i \pmod{q}\end{aligned}$$

for some $e_i = \sum_{j=0}^{i-1} e_j^{\odot} s_j + \sum_{j=0}^{i-1} \langle \mathbf{g}_r^{-1}(u_{i,j}), (\mathbf{e}_{i,1} \cdot s_j + r_i \mathbf{e}_{j,0}) \rangle + \langle \mathbf{g}_r^{-1}(v), \mathbf{e}_{i,2} \rangle$. By Lemma 1, this term is bounded by

$$((i-1)N+1) \left(\frac{dNB^2}{12} \left(\sigma_{br}^2 + d^2 \sigma_{ms}^2 + \frac{d^2-1}{3} \sigma_{ks}^2 \right) + \frac{\epsilon^2}{3} \right) + ((2i-1)N+1) \frac{d_r NB_r^2}{12} \sigma_r^2$$

for $\epsilon = \frac{q}{B^d}$. On the other hand, the above recurrence relation implies

$$\varphi_{\bar{s}}(\bar{c}_k) = X^{\sum_{j=0}^{k-1} \hat{a}_{k,j} z_{k,j}} \varphi_{\bar{s}}(\bar{c}_{k-1}) + e_k$$

$$\begin{aligned}
&= X^{\sum_{j=0}^{n-1} \hat{a}_{k,j} z_{k,j}} \left(X^{\sum_{j=0}^{n-1} (\hat{a}_{k-1,j} z_{k-1,j})} \varphi_{\bar{s}}(\bar{c}_{k-2}) + e_{k-1} \right) + e_k \\
&= X^{\sum_{i=k-1}^k \sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}} \varphi_{\bar{s}}(\bar{c}_{k-2}) + X^{\sum_{j=0}^{n-1} (\hat{a}_{k,j} z_{k,j})} e_{k-1} + e_k \\
&\vdots \\
&= X^{\sum_{i=1}^k \sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}} \varphi_{\bar{s}}(\bar{c}_0) + \sum_{t=1}^k X^{\sum_{i=t+1}^k \sum_{j=0}^{n-1} (\hat{a}_{i,j} z_{i,j})} e_t \\
&= r(X) X^{\hat{b} + \sum_{i=1}^k \sum_{j=0}^{n-1} \hat{a}_{i,j} z_{i,j}} + \sum_{t=1}^k X^{\sum_{i=t+1}^k \sum_{j=0}^{n-1} (\hat{a}_{i,j} z_{i,j})} e_t
\end{aligned}$$

Since the monomial multiplication operation solely induces coefficient permutation, the accumulated noise term $X^{\sum_{i=t+1}^k \sum_{j=0}^{n-1} (\hat{a}_{i,j} z_{i,j})} e_t$ is approximately bounded by

$$\left(\frac{k(k-1)}{2} N + k \right) \cdot \left(\frac{dNB^2}{12} \left(\sigma_{br}^2 + d^2 \sigma_{ms}^2 + \frac{d^2-1}{3} \sigma_{ks}^2 \right) + \frac{\epsilon^2}{3} \right) + (k^2 N + k) \frac{d_r N B_r^2}{12} \sigma_r^2.$$

□

C GPU Acceleration

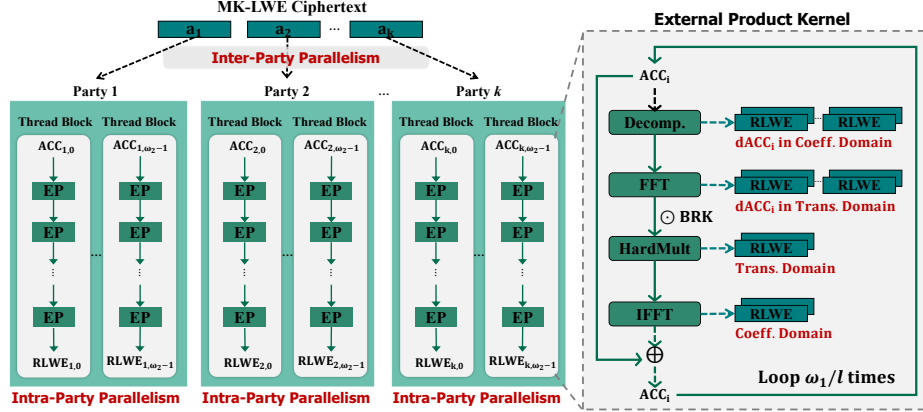


Fig. 12. Two-Level Parallel External Product Design for GPU-Based MK-LWE to RLWE Blind Rotation.

In this section, we propose the first GPU acceleration design tailored to multi-key LWE-based blind rotation. This design adopts a dual-level parallelism strategy that concurrently exploits intra-party and inter-party parallelism. We detail the GPU implementation of key execution stages, including MK-LWE to

RLWE blind rotation from block binary LWE (Layer 1, Step 1), homomorphic (Layer 1, Step 2), and hybrid ciphertext multiplication from RLEV to MK-RLWE (Layer 2).

C.1 Step 1 in Layer 1

We implement GPU-based parallel acceleration at two levels, as illustrated in Figure 12. At the *intra-party* level, we delve into operation-level parallelism by restructuring computational tasks to maximize fine-grained concurrency.

- First, we decouple the n sequential external product (EP) operations originally performed by each participant into ω_2 independent computational units. Within each unit, ω_1/l EP operations are executed sequentially using *block* LWE. This restructuring preserves computational equivalence while fully exploiting the GPU’s SIMT architecture, leading to substantial performance gains.
- Second, when mapped to the GPU architecture, independent thread blocks are allocated to the ω_2 computational units, with each thread block sequentially performing ω_1/l external products in each iteration (as shown in Lines 7-8 of Algorithm 1). By encapsulating the entire external product operation within a single kernel, we effectively minimize the overhead associated with frequent kernel launches.
- Third, to satisfy the requirement for distinct ciphertext representations (typically coefficient domain and transformation domain) at different stages of the external product computation, we architect a decomposed processing pipeline. This optimized workflow comprises four specialized operators: (1) the **Decompose** for ciphertext preprocessing, (2) the **FFT** for domain transformation, (3) the **HardMult** for point-wise modular operations with blind rotation keys, and (4) the **IFFT** for inverse transformation.

At the *inter-party* level, we formalize the parallelism as a bijective mapping $\Phi : \mathcal{P} \rightarrow \mathcal{B}$, where $\mathcal{P}$ is the set of k participants and $\mathcal{B}$ comprises $k\omega_2$ thread blocks. Each block $B_i \in \mathcal{B}$ independently computes the external product through our optimized kernel. It is important to note that before the k participants begin their computations, each component of the **a**-part in the MK-LWE ciphertext must be converted from the original modulus q to $2N$ using **modulus switching**. This conversion is handled within the external product kernel, incurring minimal computational overhead. With this parallel optimization, after completing Step 1 of Layer 1, $k\omega_2$ RLWE ciphertexts are generated concurrently.

C.2 Step 2 in Layer 1

We apply the homomorphic decode (HD) operation to the $k\omega_2$ RLWE ciphertexts, transforming them into RLEV ciphertexts, as illustrated in Figure 13. At the *intra-party* level, we further exploit the decomposed level- d structure to

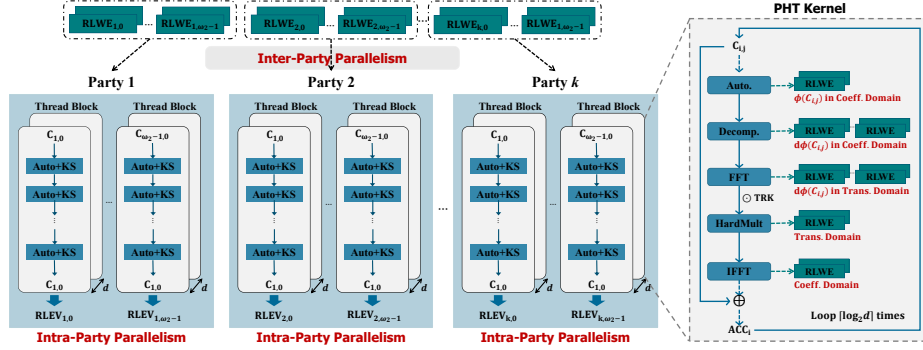


Fig. 13. Parallel Decode Acceleration with $\lceil \log_2 d \rceil$ Auto+KS Loops per Thread Block.

enable parallel acceleration. Specifically, we restructure Algorithm 3 by leveraging the hierarchical nature of level d , allowing the homomorphic automorphism and key switching operations to be executed independently and concurrently, as illustrated in Lines 3–5 of Algorithm 3. In this case, we allocate the computations corresponding to each level to individual thread blocks, where each thread block sequentially performs $\lceil \log_2 d \rceil$ Auto+KS operations, ultimately producing the updated RLWE ciphertexts. Besides, leveraging the data independence at the *inter-party* level, we assign $k\omega_2 d$ thread blocks, each responsible for executing a task at a specific decomposition level. By jointly exploiting both intra-party and inter-party parallelism, this design enables the concurrent generation of $k\omega_2$ RLEV ciphertexts. It is worth noting that the PHT kernel is responsible for both homomorphic automorphism and key switching operations. In our implementation, this involves first applying the Automorph operator to perform coefficient shift, followed by Decompose, (I)FFT, and HardMult operations using the homomorphic decode keys.

C.3 Layer 2

Focusing on the bootstrapping process in Layer 2, we exploit the parallelism available at the *intra-party* level. An analysis of Algorithm 4 reveals that the generation of each v_j by individual participants in Lines 4–6 is inherently independent. Building on this observation, we design and implement a GPU-accelerated version to fully harness this parallelism. As illustrated in Figure 14, the $k\omega_2$ RLEV ciphertexts produced in Layer 1 are transformed into MK-RLWE ciphertexts via hybrid ciphertext multiplication. As previously discussed, this step involves a generalized external product (GEP) computation (Lines 4–9 of Algorithm 4) performed over k outer iterations, where each outer iteration consists of ω_2 inner iterations that transform RLEV ciphertexts into multi-key RLWE ciphertexts. The computational workload of each GEP operation varies both across and within outer iterations. For example, in the first outer iteration ($i = 1$), the

contrast, LWE key switching is more computationally intensive but exhibits substantial *inter-party* parallelism. As shown in Figure 15, we assign key switching operations to separate thread blocks, enabling concurrent execution across multiple parties and significantly improving overall throughput. In our implementation, we preload the key switching keys into global memory according to the selected decomposition base B . This space-for-time optimization achieves better performance compared to real-time vector-matrix multiplication, since the total memory footprint of the key switching keys typically remains within 1 GB, even under aggressive parameter choices such as a decomposition base of 8. During the LWE key switching process, only the extracted $\mathbf{a}$ -component is decomposed. The corresponding key switching keys are retrieved based on the decomposition indices and then accumulated to update $\mathbf{a}$. As a result, a multi-key LWE ciphertext of the updated $(b, \mathbf{a}_1, \dots, \mathbf{a}_k)$ is obtained.

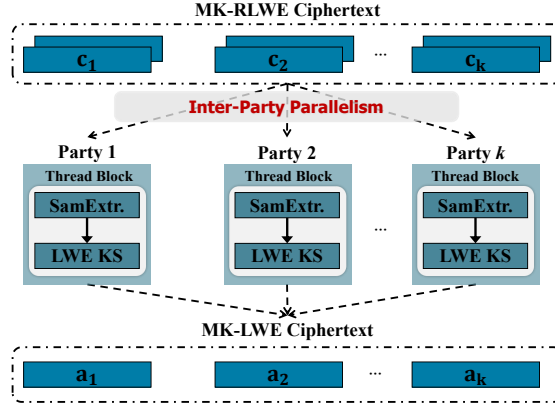


Fig. 15. LWE Key Switching Acceleration with Inter-party Parallelism.